

Sistema de Detección de Sentimiento en Reseñas de Usuarios (DSR-AI)

Memoria del proyecto

Universitat Politècnica de València

Grado en Inteligencia Artificial

Proyecto I. Introducción a la IA

Adrián A. Acosta Villegas

Sergio Garfella Pérez

Ainara Sanfélix Ruiz

Jairo E. Urdaneta Colmenares

19 de mayo de 2026

Índice

Parte I. Definición de proyecto	5
1. Contexto del proyecto	6
2. Objetivo del proyecto.....	6
3. Interesados del proyecto (stakeholders)	7
4. Necesidades, especificaciones y requerimientos del proyecto	8
5. Desglose del proyecto en tareas	10
6. Ciclo de vida de los datos	11
7. Planificación del proyecto.....	13
Diagramas de Gantt	13
Diagrama PERT	15
8. Seguimiento del proyecto	16
9. Reflexión a posteriori	18
Diagramas de Gantt y PERT	18
Reflexión y notas	21
Parte II. Análisis exploratorio de datos	22
1. Accesibilidad y Preparación de los Datos	23
Descripción de los Datasets.....	23
Unificación de los datasets	23
Resultados de la preparación	24
2. Análisis Univariante.....	25
Distribución de Sentimiento	25
Distribución de Longitud de Reseñas	25
Análisis de Vocabulario y Ley de Zipf	29
Decisiones a realizar	31
3. Análisis Multivariante	32
Longitud vs Sentimiento	32
Correlación entre las variables	32
Test de Mann-Whitney U.....	33
Vocabulario Significativo	34
Frecuencias absolutas	34
Log-Odds Ratio	35
Prueba de relevancia	36
Análisis de pares y tríos de palabras.....	38

4. Evaluación de Supuestos	39
Normalidad de las Variables Numéricas	39
Linealidad entre variables.....	41
Homogeneidad de Varianzas.....	42
5. Detección de Valores Atípicos (Outliers)	42
Detección mediante IQR y Z-Score	42
Magnitud vs Apalancamiento	43
Reseñas con Caracteres Anómalos	44
Reseñas Extremas.....	44
Reseñas con Caracteres Anómalos	45
Reseñas Extremas.....	45
Decisión sobre el tratamiento de outliers	46
Valores ausentes (missing)	46
6. Conclusiones del análisis exploratorio de datos.....	47
Parte III. Desarrollo técnico y despliegue	48
1. Entrenamiento del modelo	49
K-Nearest Neighbors (k-NN).....	49
Flujo de Preparación y Carga de Datos	49
Entrenamiento.....	50
Evaluación.....	50
Configuración del Modelo (Hiperparámetros)	50
Regresión Logística.....	52
Flujo de Carga y Consistencia de Datos	52
Configuración del Modelo (Hiperparámetros)	52
Entrenamiento.....	53
Evaluación.....	53
Resultados de nuestros modelos	55
2. Flujo de procesamiento de texto	56
1. Traducción Universal.....	56
2. Limpieza y normalización.....	56
3. Vectorización numérica.....	57
4. Clasificación y predicción	57
3. Interfaz: Capturas y explicación del frontend	58
desarrollado.	58
Arquitectura del Sistema	58

Análisis de la Interfaz (Frontend)	59
Pruebas de nuestra aplicación completa	60
4. Despliegue de la aplicación	64
Paquete de la app	64
Docker	64
DockerHub	65
Referencias	66
Anexos	66

Parte I. Definición de proyecto

Definiremos las bases de la organización y estructura de nuestro desarrollo. Definiremos los objetivos principales del sistema, el rol de cada uno de los integrantes del equipo, el alcance de lo que vamos a entregar y las herramientas de gestión que utilizaremos para coordinar nuestras tareas semanales

1. Contexto del proyecto

Somos un grupo de estudiantes universitarios realizando un proyecto para la asignatura Proyecto I del Grado en Inteligencia Artificial. En este trabajo nos enfrentamos a una situación real planteada por la empresa Meta.

Meta desea comprender mejor la percepción que los usuarios tienen sobre productos, restaurantes y películas. Actualmente, en muchas ocasiones dicho análisis de reseñas se sigue realizando de forma manual o mediante recuentos simples de palabras, dificultando la obtención de conclusiones fiables cuando el volumen de opiniones es elevado. La organización dispone de conjuntos de datos con reseñas escritas por usuarios y desea explorar el uso de técnicas de aprendizaje automático para identificar automáticamente si una reseña expresa un sentimiento positivo o negativo. Para mejorar la presentación semántica del texto, se propone utilizar modelos de vectores de palabras y documentos, específicamente el modelo Doc2Vec, junto con un clasificador basado en K-Nearest Neighbors (k-NN), algoritmo de vecinos cercanos.

2. Objetivo del proyecto

Desarrollar un modelo en Python capaz de clasificar las reseñas de los usuarios como positivas o negativas. El texto se representará en la aplicación utilizando Doc2Vec, y se utilizará un algoritmo de k-NN como método de clasificación. Los resultados deben ser entregados de forma comprensible y sencilla para usuarios no expertos en técnicas de inteligencia artificial (gente de marketing digital, publicidad, directivos), mediante una interfaz sencilla y amigable. El sistema pretende servir como herramienta de apoyo para el análisis de opiniones y la toma de decisiones basada en datos.



3. Interesados del proyecto (stakeholders)

Interesados	Rol en la institución	¿Que necesita del sistema?	¿Que valor obtiene?
Comprador/Usuario final	Usuario consumidor	Un análisis de un sentimiento que genera en usuarios sobre un servicio o producto	Una mejor visión de la opinión general, lo que le permite mejores decisiones de compra.
Vendedor/Propietario	Usuario proveedor	Un análisis de las tendencias que producen emociones favorables y conocer el sentimiento generado por las ventas o servicios de su negocio/producto.	Información de ayuda en la decisión de nuevos productos a incorporar (nuevas oportunidades). También evaluar el desempeño de sus productos.
Creadores de contenido	Usuario Creador	Un análisis de las emociones que produce su contenido.	Conocer qué tipo de contenido obtienen mejor recepción para futuras mejoras.
Equipo de Meta	Marketing/Publicidad	Conocer los intereses de los usuarios según el contenido que consumen y las emociones que produce este contenido	Obtención de información para poder ofrecer una publicidad más personalizada para el usuario.
Equipo de Meta	Analistas de datos/Desarrolladores	Conocer los intereses de los usuarios según el contenido que consumen y las emociones que produce este contenido	Utilización de la información recopilada para que, mediante un algoritmo, se recomienden al usuario vídeos con mayor probabilidad de interés.
Equipo de Meta	Directivos	Conocer los intereses de los usuarios según el contenido que consumen y las emociones que produce este contenido	Obtención de información y análisis del consumidor para la toma de decisiones de la empresa.

4. Necesidades, especificaciones y requerimientos del proyecto

El director de operaciones de Meta se ha puesto en contacto con vosotros para ofreceros la siguiente información:

Para ofrecer una solución al problema planteado, los datos de entrada serán textos de reseñas en formato textual, almacenados por la propia empresa. La aplicación debe clasificar automáticamente el sentimiento expresado por cada uno de ellos. Para ello, se debe manejar un lenguaje natural realista, incluyendo variaciones en estilo y vocabulario, pudiendo clasificar el mayor amplio espectro de comentarios. En la aplicación final se debe poder introducir un comentario y obtener su clasificación como positivo o negativo. Dicha aplicación debe poder ejecutarse en un entorno estándar sin requerimientos complejos de infraestructura (como un ordenador de con un sistema operativo *Windows* instalado). Para ello, la dirección sugiere el uso de una pequeña interfaz web utilizando *Streamlit*.

El proyecto en su totalidad engloba la comprensión y análisis del problema y sus requisitos. Los datos proporcionados deberán ser explorados, tratados y preparados para ser usados en el modelo de clasificación. Se definirá el modelo a utilizar, se entrenará con los datos proporcionados y se evaluarán los resultados del mismo hasta que las predicciones se ajusten con los datos históricos. El modelo entrenado será incorporado en una aplicación sencilla que permita el uso por parte del cliente, e irá acompañado de documentación técnica y funcional.

El acuerdo del proyecto no incluye ni la integración de la aplicación con sistemas reales de comercio electrónico o plataformas de reseñas, ni el despliegue de la misma a gran escala.

Listado de requisitos, necesidades y alcance del proyecto



Requisitos y necesidades de los interesados

- **Datos de entrada:** Procesar textos de reseñas en formato de texto que ya están almacenados por la empresa.
- **Procesamiento de Lenguaje Natural (NLP):** Capaz de manejar un lenguaje natural realista, incluyendo variaciones en el estilo y el vocabulario, para poder clasificar un amplio espectro de comentarios.

- **Clasificación automática:** Clasificar de forma automática el sentimiento de cada reseña introducida como positivo o negativo.
- **Interfaz de usuario:** Desarrollo de una pequeña interfaz web utilizando la librería *Streamlit* donde el usuario pueda introducir un comentario y obtener su clasificación.
- **Infraestructura:** Ejecución en un entorno estándar (como un ordenador con sistema operativo *Windows*), sin necesitar una infraestructura compleja.
- **Documentación:** Entrega junto con su correspondiente documentación.

✓ Alcance del proyecto

- Comprensión y análisis del problema y de los requisitos planteados.
- Exploración, tratamiento y preparación de los datos para su uso en el modelo.
- Definición, entrenamiento y evaluación del modelo de clasificación hasta lograr que las predicciones se ajusten correctamente a los datos históricos.
- Incorporación del modelo dentro de la aplicación final para que pueda ser utilizada por el cliente.

✗ Límites del proyecto

- No se realizará la integración de la aplicación con sistemas reales o plataformas de reseñas.
- No se realizará un despliegue de la aplicación a gran escala.



5. Desglose del proyecto en tareas

Realizamos un primer desglose del proyecto en tareas específicas. Es importante contemplar posibles variaciones y mantener la flexibilidad necesaria para garantizar una ejecución óptima del proyecto.

Las tareas más cercanas tienen más detalle, control y certeza en la forma y tiempo de ejecución.

Fase 1

(3-Feb a 3-Mar)

Planificación

- Distribución inicial de tareas
- Creación de repositorio publico
- Planteamiento de cronograma y time-line del proyecto

Análisis de los requisitos

- Definición del proyecto: objetivos, requisitos, arquitectura
- Creación del entregable
- Revisión entregable
- Presentación entregable

Fase 2

(10-Mar a 14-Abr)

Diseño/Codificación

- Recolección de datos a utilizar
- Comprobación de los datos
- Desarrollo del programa
- Documentación de progreso

Fase 3

(21-Abr a 19-May)

Pruebas

- Entrenamiento y testing del modelo
- Documentar resultados
- Revisar y eliminar errores del codigo

Lanzamiento

- Implementación de la interfaz gráfica, user-friendly
- Presentación del proyecto



6. Ciclo de vida de los datos

El ciclo de la vida se dividirá en las siguientes fases:



Captura

Dado que contamos con *datasets* sobre análisis de sentimientos relacionados con diversas películas, proporcionados por la Universidad de Stanford y por la Universidad de Cornell, no será necesario recopilar datos desde cero. El proceso consistirá en la extracción de información, ya sea accediendo a los archivos disponibles en el repositorio digital abierto de Meta o importándolos mediante una librería de *Python*. Cada archivo incluye tres conjuntos de datos distintos sobre reseñas de películas, con variaciones de estilo y vocabulario diseñadas para reflejar el lenguaje natural.



Almacenamiento

Contamos con tres *datasets* en formato `.txt` que contienen reseñas de usuarios, clasificadas en positivas y negativas. Dado que se trata de datos no estructurados, es óptimo utilizar bases de datos no relacionales disponibles en la nube, como Word2Vec. Esta elección permite un manejo más eficiente del lenguaje natural y facilita la integración de herramientas de procesamiento de texto sin las limitaciones de un esquema rígido.



Preprocesado

Uniremos los tres *datasets* en uno solo y, a continuación, crearemos una jerarquía para clasificar los atributos del conjunto de datos. Posteriormente, realizaremos una limpieza del dataset, eliminando ruido e inconsistencias, normalizando el texto a minúsculas, eliminando stopwords y aplicando tokenización (división del texto en palabras individuales).



Análisis (Minería de datos)

Crearemos un modelo que explique las características de los datos.

Emplearemos un enfoque de aprendizaje supervisado, que nos permitirá identificar patrones lingüísticos a partir de datos previamente etiquetados. Asimismo, desarrollaremos un clasificador basado en K-Nearest Neighbors (k-NN), que asignará una categoría a cada reseña según su cercanía vectorial con ejemplos ya clasificados, con el objetivo de determinar si el sentimiento es positivo o negativo.

Nuestro propósito será identificar automáticamente el sentimiento y obtener conclusiones fiables frente a grandes volúmenes de reseñas.



Visualización

Representamos los resultados para facilitar la extracción de conocimiento.

Se utilizará una interfaz web llamada Streamlit, diseñada para ser intuitiva y accesible para personal no experto en inteligencia artificial. La interfaz permitirá introducir reseñas en lenguaje natural y obtener de forma inmediata su clasificación como positiva o negativa.

El sistema facilitará tareas básicas, como el filtrado de opiniones irrelevantes y la obtención de detalles bajo demanda sobre grupos específicos de reseñas.

El objetivo es proporcionar una herramienta de apoyo que simplifique la comprensión de grandes volúmenes de datos, permitiendo una toma de decisiones basada en la percepción real del usuario.



Publicación

Documentaremos el *dataset* creado para facilitar su utilización en futuros proyectos.

Posteriormente, se ejecutará una preservación en un repositorio digital en la nube para asegurar la disponibilidad, integridad y seguridad de los datos a largo tiempo.



Preservación

El objetivo es garantizar la conservación a largo plazo de los datos y del modelo desarrollado.

Para ello, se priorizará el archivo de las reseñas y de las versiones del modelo más relevantes, se aplicará un proceso de estandarización para facilitar su uso por otros departamentos y se emplearán formatos compatibles como JSON o CSV. Además, se gestionarán adecuadamente las licencias y la privacidad, y se acompañarán los archivos con documentación que describa la metodología y los parámetros utilizados.



Archivo

Cuando los datos pierden su relevancia operativa inmediata, se pasan a una fase de archivo para su consulta histórica o cumplimiento legal. Debido a esto se debe establecer cuando se archiven los datos, donde se almacenarán y durante cuánto tiempo permanecerán archivados.



Eliminación

Cuando se cumpla la etapa de retención, se procederá a la eliminación definitiva del *dataset*. En este proceso se tendrá que eliminar los datos de manera segura, para que estos datos no puedan ser reconstruidos. Esta es una fase crítica para el cumplimiento de la privacidad y la optimización del almacenamiento usado.

7. Planificación del proyecto

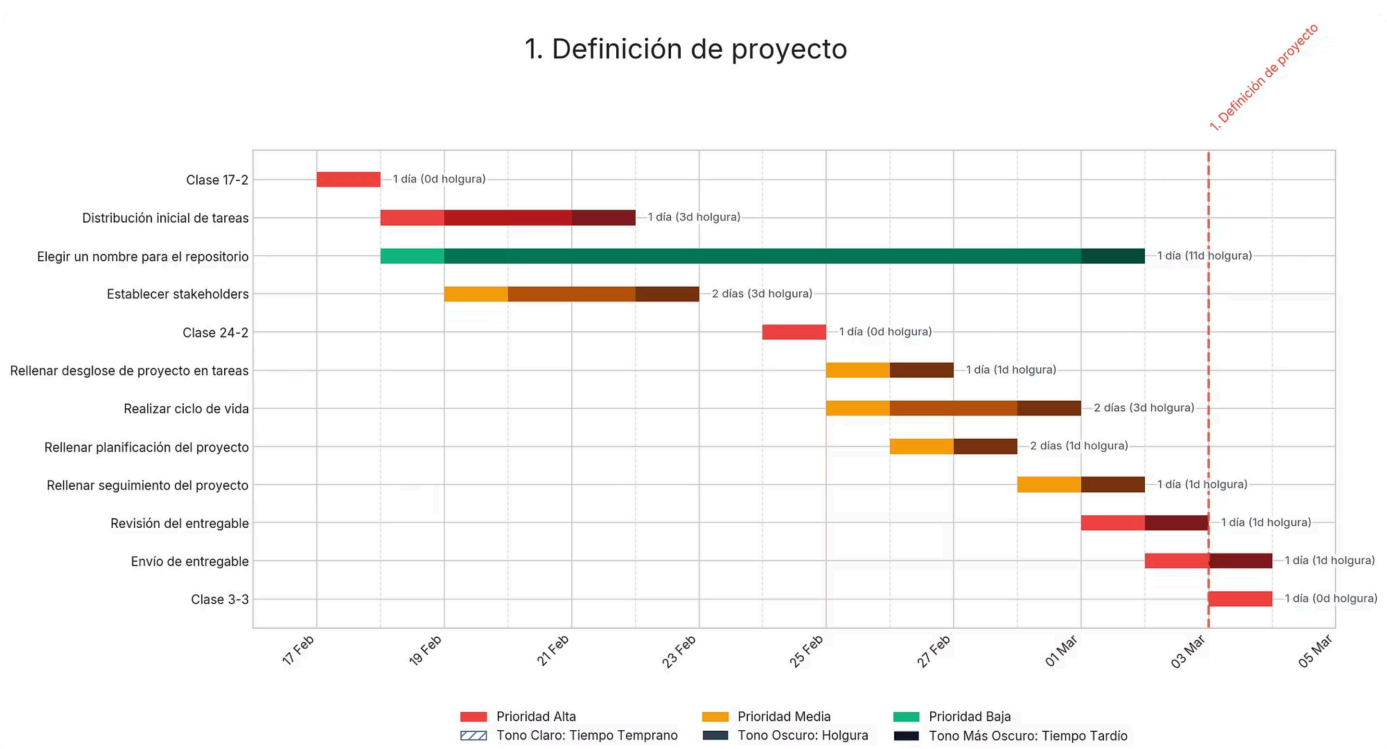
Realizaremos toda la planificación del proyecto mediante [GitHub](#). Esta plataforma nos permite sincronizar las tareas en *issues* y realizar una planificación del proyecto con diagramas en tiempo real. Además podemos relacionar las *issues* con los *commit* que actualizan el [repositorio](#).

Hemos desarrollado un script en *Python*, con ayuda de Gemini, que permite generar de forma automatizada diagramas de PERT y de Gantt a partir de la planificación del proyecto desde la API de GitHub. Está ubicado en el directorio `/tools/generador-diagramas`

El objetivo de esta herramienta es facilitar la visualización de dependencias, tiempos estimados y ruta crítica (en el caso del PERT), así como la distribución temporal de tareas y su seguimiento (en el caso del Gantt). De este modo, tenemos una visión del avance del proyecto.

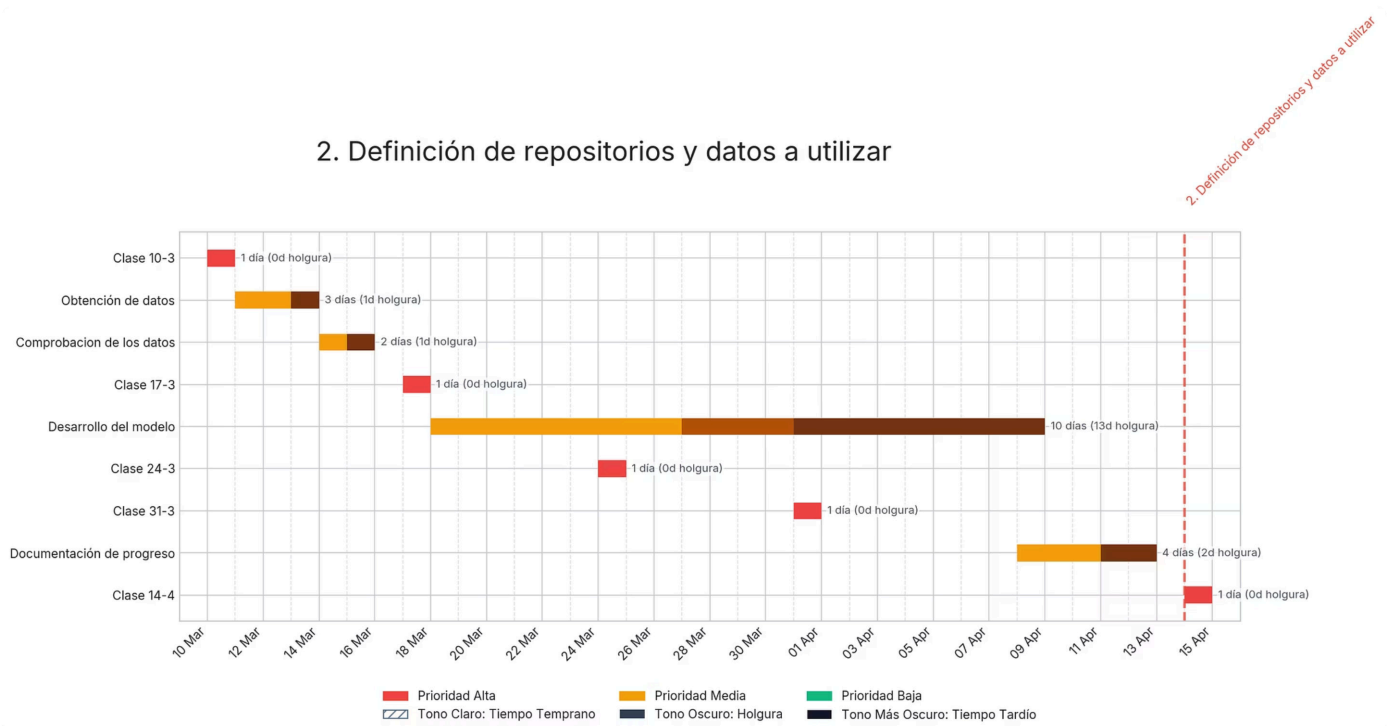
Diagramas de Gantt

En el siguiente gráfico se muestra el diagrama de Gantt del primer sprint. Al tratarse de un periodo cercano en el tiempo, la planificación es bastante precisa y las estimaciones son más ajustadas.



Además, hemos elaborado una primera versión de los diagramas del segundo y tercer sprint. Aunque aún pueden sufrir cambios, nos sirven como guía para visualizar cómo se distribuirán las tareas más adelante y para hacernos una idea del ritmo y la organización del trabajo en las próximas etapas.

2. Definición de repositorios y datos a utilizar



3. Código y memoria

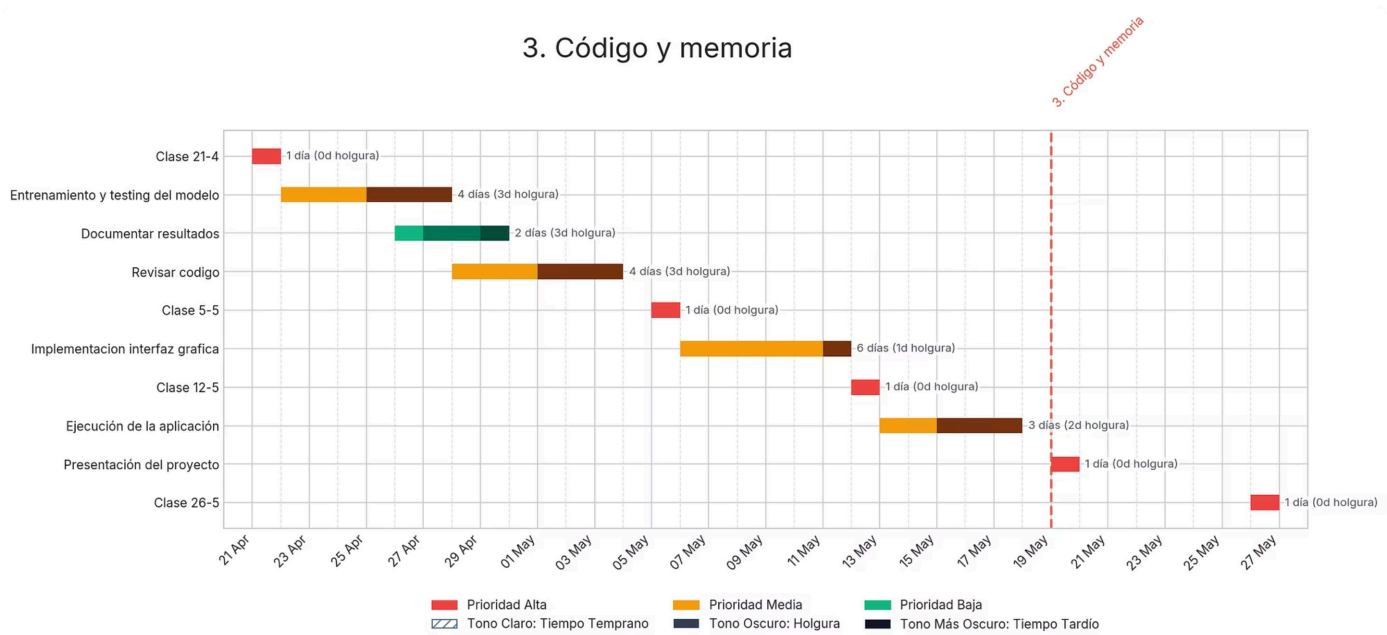
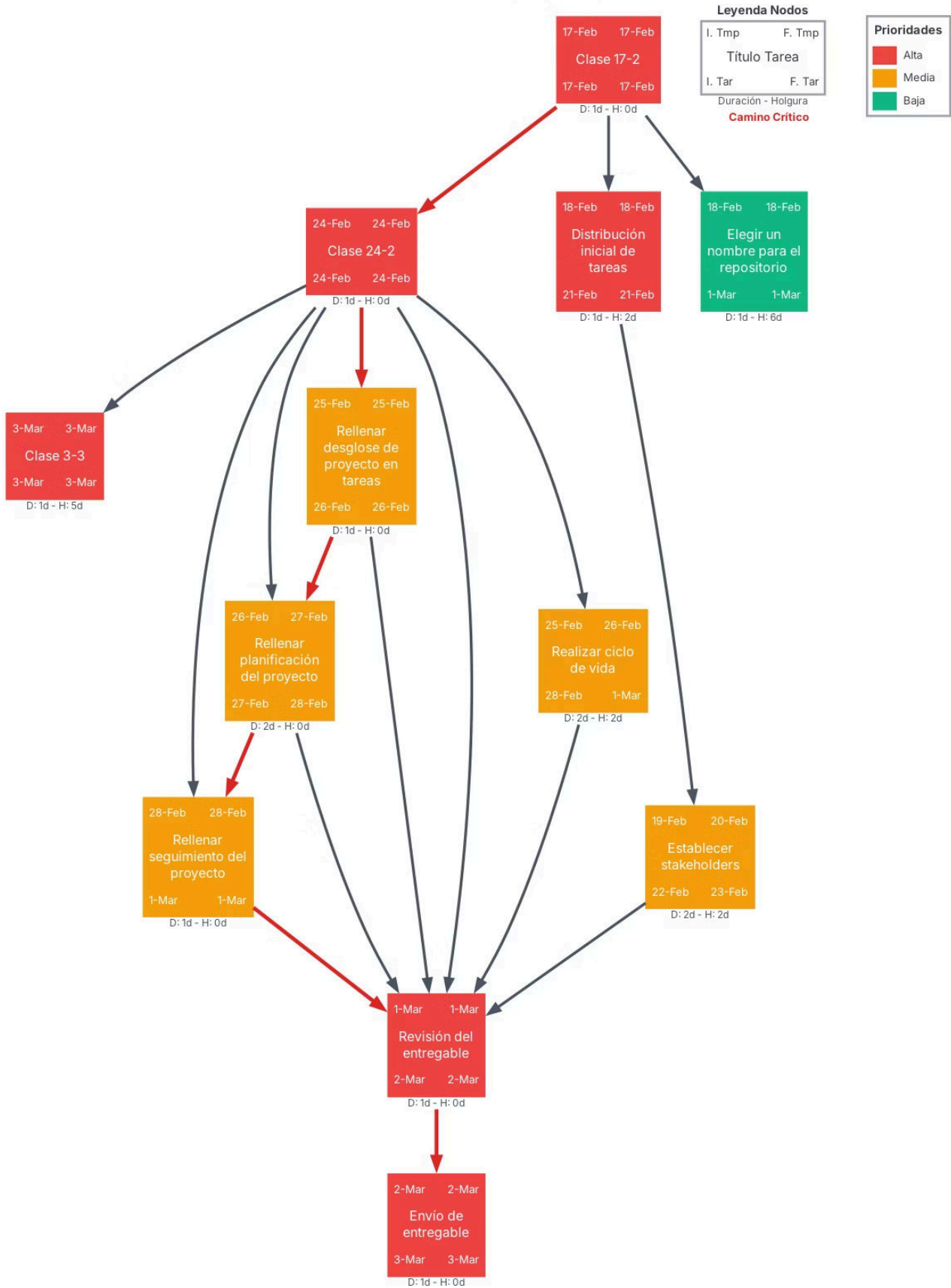


Diagrama PERT

Representamos el diagrama PERT de las tareas planificadas para el primer sprint. En él se muestran los tiempos de inicio y fin tanto de tempranos como tardíos, así como el camino crítico del sprint.

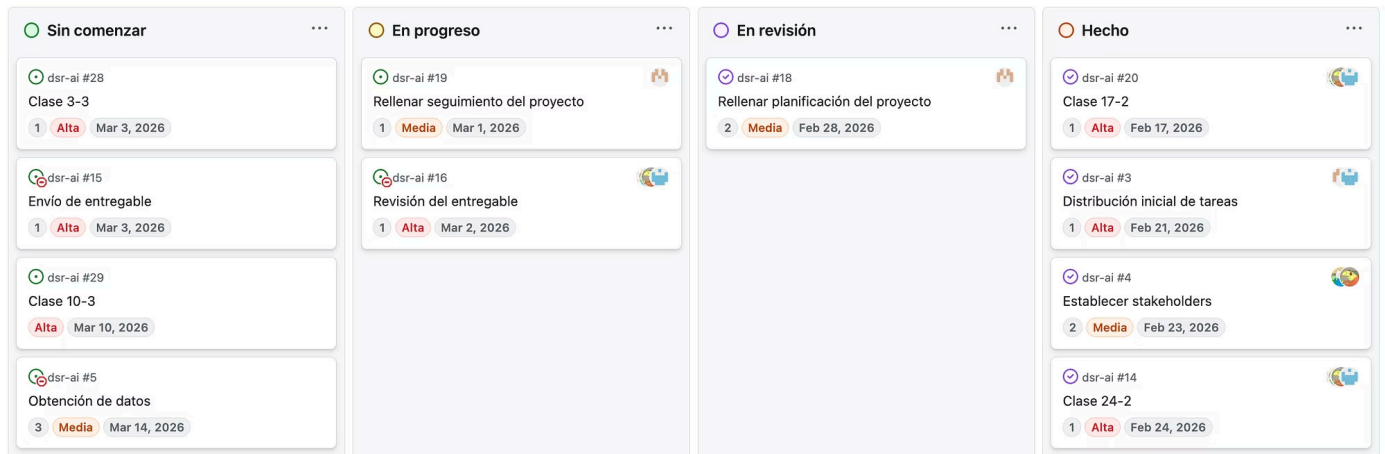
1. Definición de proyecto



8. Seguimiento del proyecto

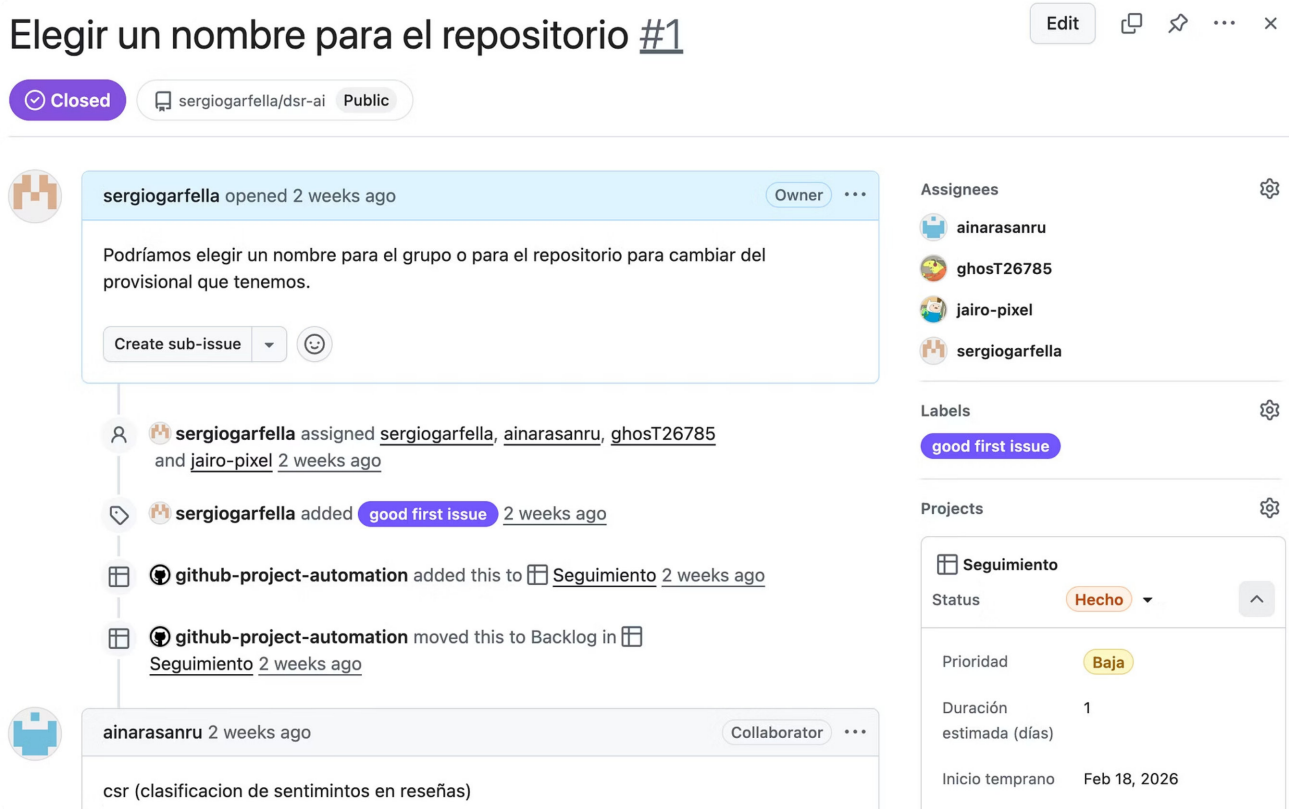
Al igual que la planificación, realizaremos el seguimiento del proyecto en [GitHub](#). Podremos establecer, actualizar y auditar cada *issue* estableciendo el desarrollador responsable, importancia, fechas límite, etc.

Utilizamos una tabla *Kanban* para visualizar el progreso de las tareas. A continuación vemos un ejemplo:



En cada tarea podemos ver las personas que están asignadas, la duración (días), la prioridad y la fecha límite (final tardío).

Si entramos en el *issue* podemos ver toda la información detallada:



Reuniones

Programaremos reuniones periódicas para debatir y consensuar el progreso del trabajo. Estas podrán celebrarse de forma presencial o en línea.

En cada reunión se elaborará un acta que recoja los puntos tratados, las decisiones consensuadas y las tareas asignadas. Todas las actas se subirán al [repositorio](#) en el directorio docs/reuniones/.

Por ejemplo, el acta de la reunión 1 tiene el siguiente formato:



Acta de Reunión 1: Arranque y Entorno de Trabajo

Fecha: 17/02/2026

Asistentes:

- Adrián A. Acosta Villegas
- Sergio Garfella Pérez
- Ainara Sanfélix Ruiz
- Jairo E. Urdaneta Colmenares.

Orden del día:

- Organización e infraestructura del proyecto
- Planificación de tareas
- Sistema de seguimiento del desarrollo

Desarrollo de la reunión

Durante la reunión se debatió sobre la infraestructura y la organización del proyecto, acordándose centralizar toda la planificación mediante la plataforma *GitHub*.

Se optó por emplear los issues del repositorio para asignar tareas, organizar prioridades y establecer fechas de entrega.

Además, se decidió que el progreso se registrará vinculando cada issue con los commits correspondientes en el repositorio público.

9. Reflexión a posteriori

Diagramas de Gantt y PERT

Tras finalizar todas las fases del proyecto nuestros diagramas fueron los siguientes:

1. Definición de proyecto

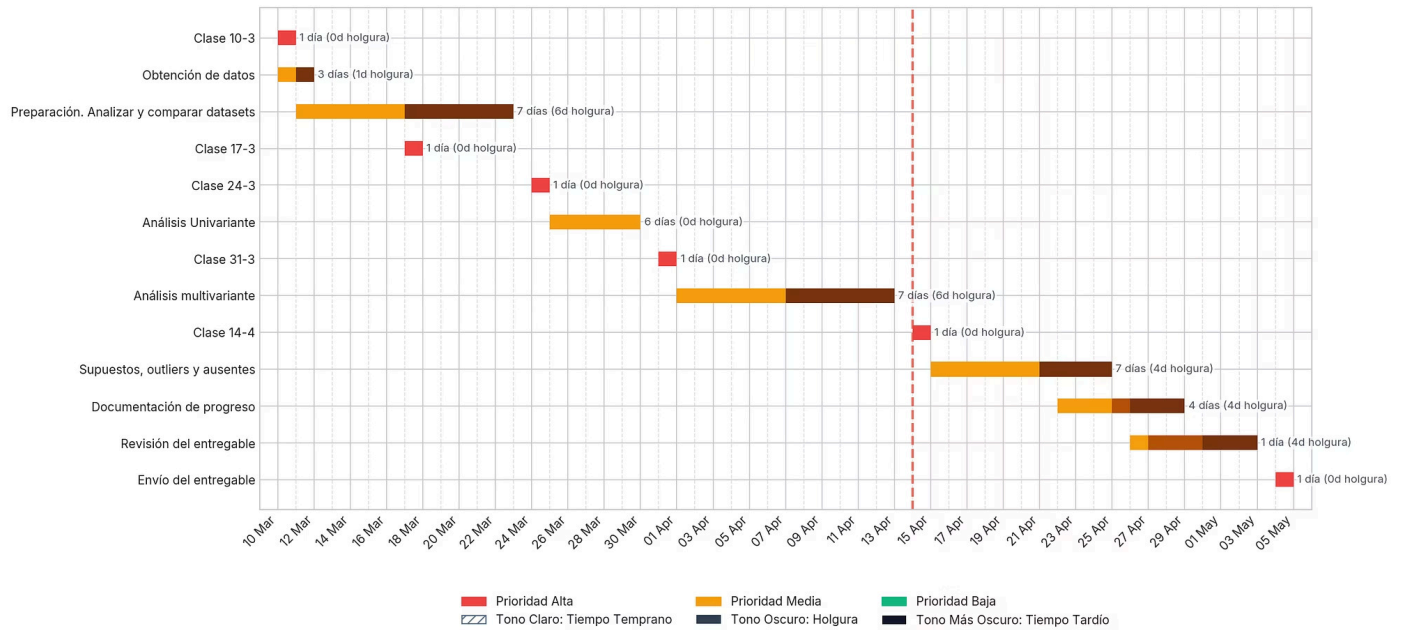


1. Definición de proyecto

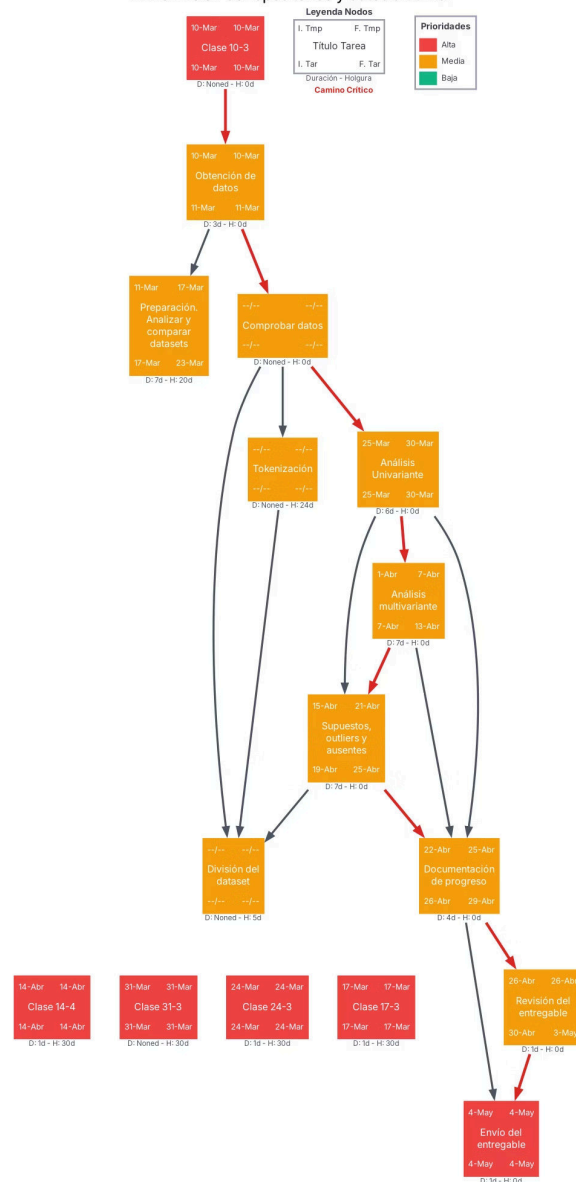


2. Definición de repositorios y datos a utilizar

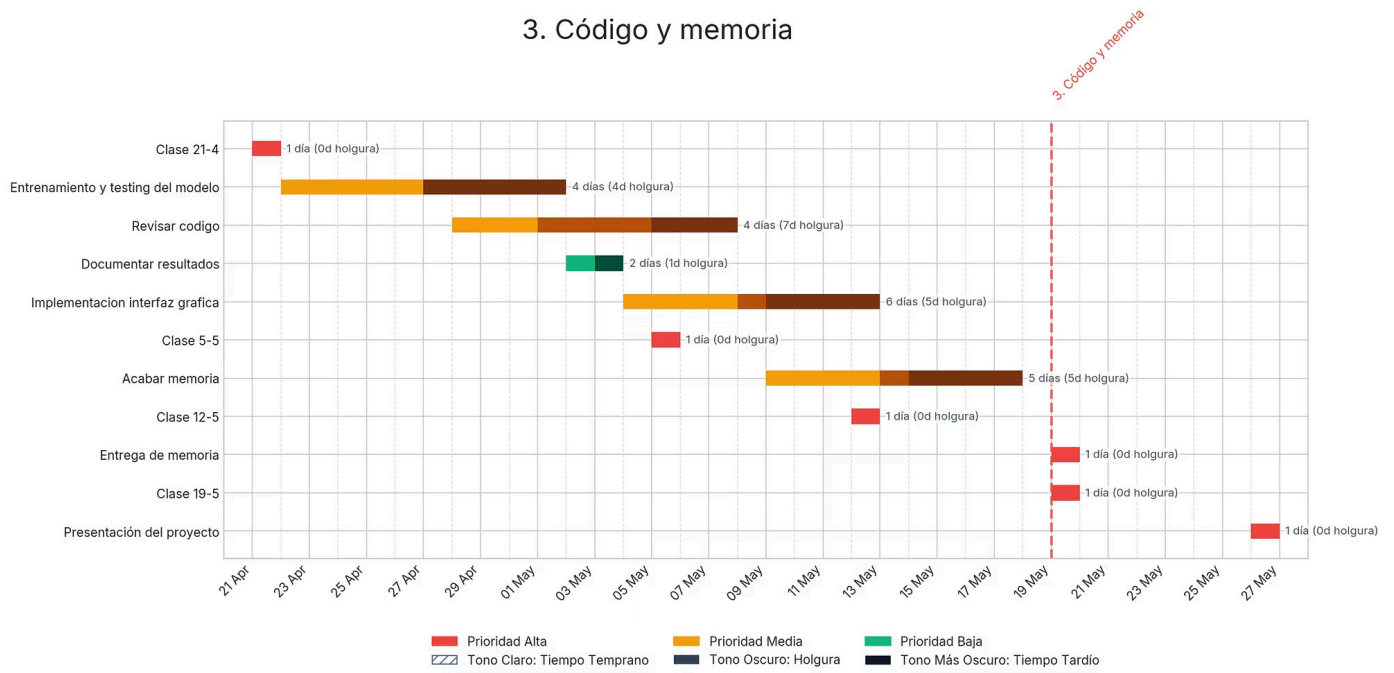
2. Definición de repositorios y datos a utilizar



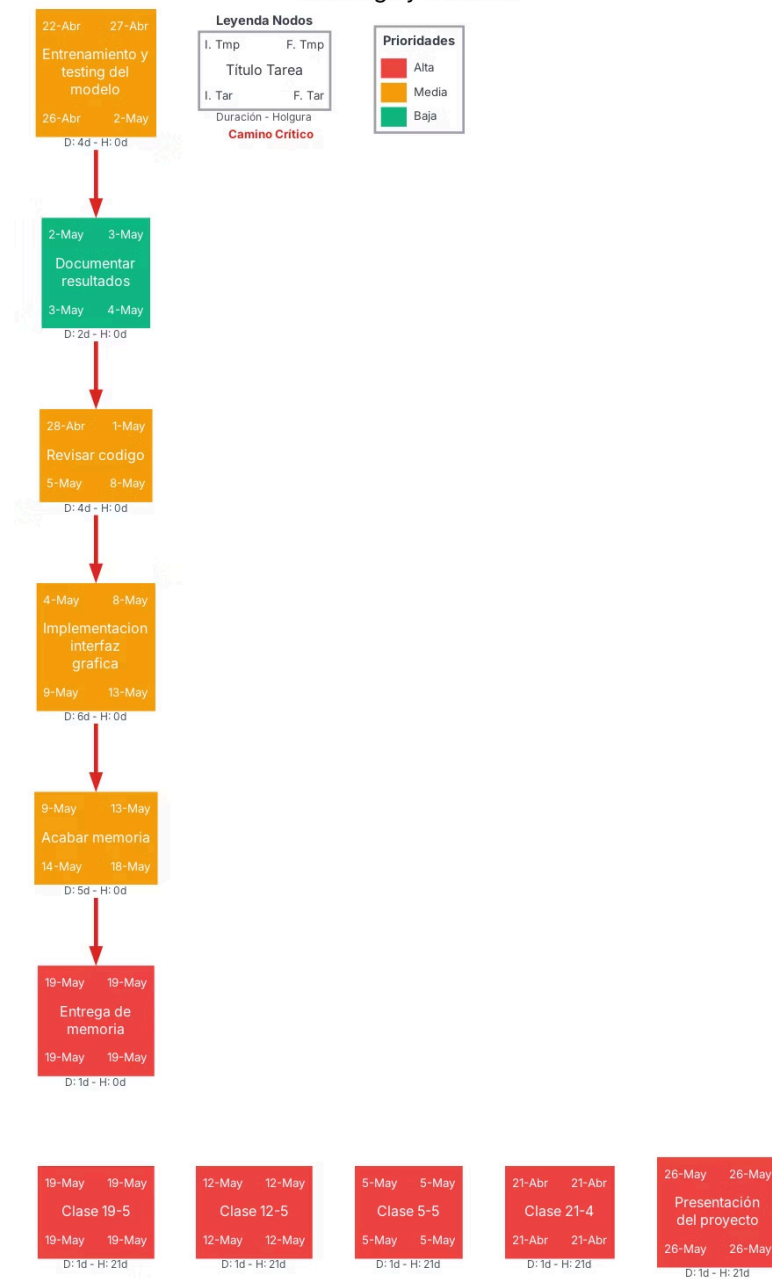
2. Definición de repositorios y datos a utilizar



3. Código y memoria



3. Código y memoria



Reflexión y notas

A lo largo del proyecto, nos dimos cuenta que lo más importante para avanzar es la actitud y colaboración entre los miembros del grupo. Debido a otros exámenes y entregas, tuvimos que trabajar más en *sprints* que de forma continua, pero el equipo respondió con compromiso. Cuando hizo falta, todos dimos el 100% para sacar el proyecto adelante de manera colaborativa.

También decidimos dejar de hacer reuniones formales con actas, ya que no aportaban tanto como el trabajo que costaba hacerlas. Al vernos entre semana, podíamos hablar en persona, resolver dudas rápidamente y organizarnos de forma natural como equipo. Cuando necesitábamos conectarnos online, utilizamos WhatsApp por su rapidez para mantenernos coordinados.

Consideramos que la realización de este proyecto no solo nos ha hecho aprender el desarrollo técnico de un proyecto de IA, sino que nos ha hecho mejorar en habilidades claves como la gestión de equipos y comunicación.

Parte II. Análisis exploratorio de datos

Haremos un estudio de los conjuntos de datos para comprender su estructura y características.

Realizamos un análisis univariante y multivariante para evaluar la distribución del sentimiento, la longitud de las reseñas y la riqueza léxica del vocabulario. También comprobaremos supuestos estadísticos y aplicaremos técnicas de detección de valores atípicos para depurar el corpus.

1. Accesibilidad y Preparación de los Datos

Descripción de los Datasets

Para el desarrollo del sistema de detección de sentimiento usamos tres conjuntos de datos de referencia en el ámbito del análisis de sentimiento de reseñas cinematográficas. Cada uno tiene características distintas en cuanto a volumen, estructura y nivel de detalle, lo que enriquece la diversidad del corpus de entrenamiento.

Dataset	Fuente	Formato	Clasificación	Registros etiquetados
acllmbd	Stanford (Maas et al., 2011)	Archivos TXT	Binaria (pos/neg)	50.000
review_polarity	Cornell (Pang & Lee, 2004)	Archivos TXT	Binaria (pos/neg)	2.000
standfordSTT	Stanford (Socher et al., 2013)	Archivos TSV	5 clases) → Binaria	11.286

El dataset **acllmbd** es el más extenso, con 50.000 reseñas etiquetadas (25.000 positivas y 25.000 negativas) divididas a su vez en conjuntos de entrenamiento (25.000) y test (25.000). Cada archivo lleva en su nombre una puntuación original del 1 al 10. El *dataset* **review_polarity** contiene 2.000 reseñas (1.000 positivas y 1.000 negativas) organizadas en 10 particiones de validación cruzada (cv000 a cv009). El *dataset* **standfordSTT** tiene 11.855 frases con puntuaciones de sentimiento continuas en el rango [0; 0,2] (muy negativo) a [0,8; 1,0] (muy positivo), con una estructura de árbol sintáctico que permite análisis a nivel sub-frase.

Unificación de los *datasets*

Para poder trabajar con los tres orígenes de forma conjunta, hemos realizado el siguiente proceso de unificación.

1. Carga e inspección: Cargamos cada *dataset* en su formato particular.

2. Conversión a binario (STT): Al ser el único *dataset* con sentimiento continuo, hemos aplicado un umbral para convertir la puntuación [0,0; 1,0] a etiqueta binaria. Hemos establecido: puntuación $\geq 0,6 \rightarrow$ **positivo**, puntuación $\leq 0,4 \rightarrow$ **negativo**. Las frases con puntuación en el rango (0,4; 0,6) las hemos descartado por ser de sentimiento neutral o ambiguo.

3. DataFrame unificado: Hemos creado un DataFrame común con las columnas: texto (contenido de la reseña), sentimiento (pos/neg), dataset (origen) y split (train/test/cv/dev).

4. DataFrame unificado con análisis: Para facilitar el análisis añadiremos al DataFrame anterior variables derivadas del texto: longitud, vocabulario y número de caracteres.

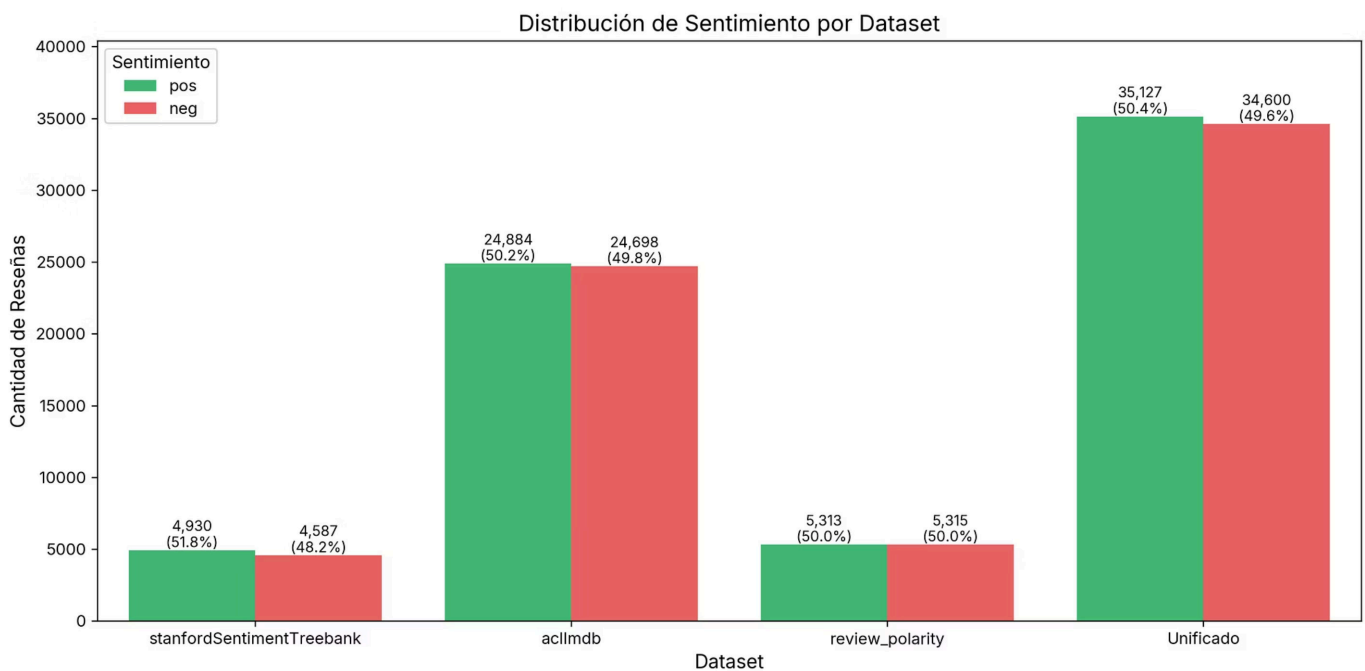
Resultados de la preparación

Métrica	Valor
Registros originales (3 datasets)	72.601
Registros etiquetados (pos/neg)	70.327
Reseñas vacías eliminadas	0
Duplicados eliminados	600
Stanford neutrales descartados ($0,4 < \text{score} < 0,6$)	2.274
Dataset unificado	69.727

2. Análisis Univariante

Distribución de Sentimiento

La variable objetivo del problema es el sentimiento de la reseña, codificado como positivo o negativo. A nivel global, el conjunto tiene una distribución prácticamente equilibrada, con un 50,4% de reseñas positivas (35.127) frente a un 49,6% de negativas (34.600). Este equilibrio es gracias al diseño de los datasets originales, que fueron contruidos con representación equitativa de ambas clases.

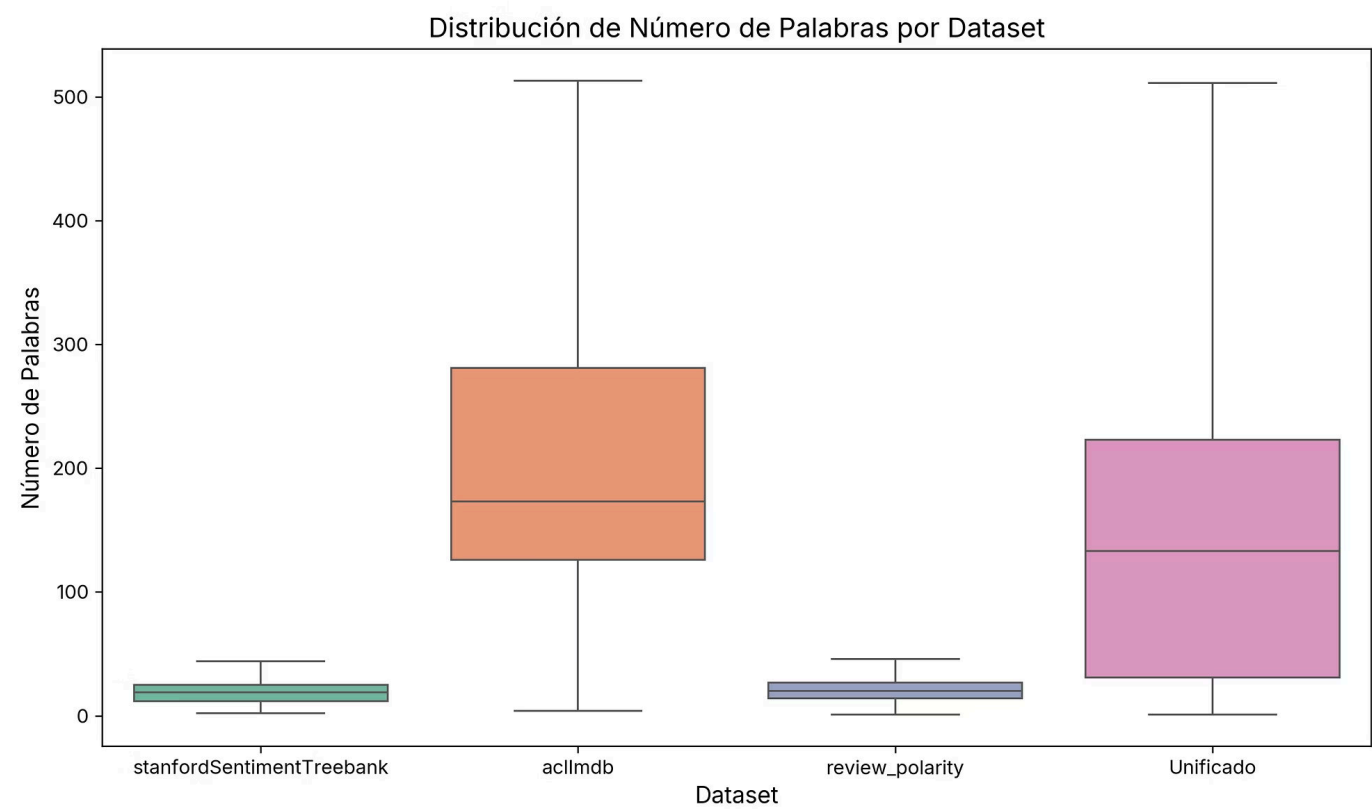


Visualizando los gráficos de distribución de cada *dataset*, llegamos a la conclusión de que el más equilibrado es **review_polarity**, con una distribución totalmente equilibrada de 50% reseñas positivas frente a un 50% de reseñas negativas. Seguido de **aclImdb** con un 50,2% de reseñas positivas y un 49,8% de negativas. Por último **stanfordSST** que cuenta con un 51,8% de positivas y 48,2% de negativas.

Distribución de Longitud de Reseñas

Hemos analizado tres medidas de longitud: Número de palabras, número de caracteres y número de frases.

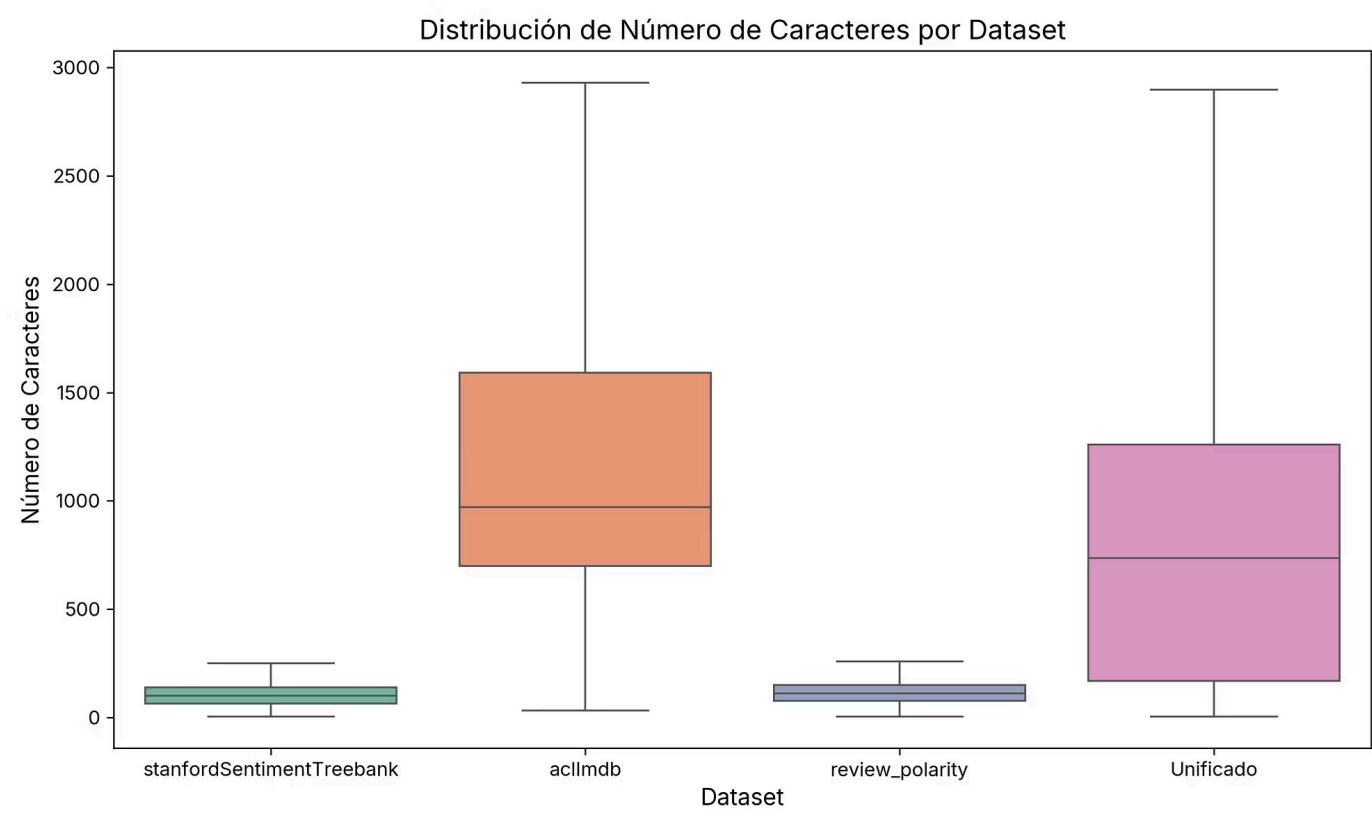
Distribución de palabras



	acllmbd	review polarity	stanfordSTT	Unificado
Media	231,4	21,0	19,3	170,4
Std	171,5	9,4	9,2	173,5
Mínimo	4,0	1,0	2,0	1,0
P25	126,0	14,0	12,0	31,0
Mediana	173,0	20,0	19,0	133,0
P75	281,0	27,0	25,0	223,0
P95	591,0	38,0	36,0	523,0
Máximo	2470,0	59,0	56,0	2470,0

La distribución de palabras muestra grandes diferencias y asimetrías: **acllmbd** tiene una gran variedad y extensión (mediana de 173 palabras), mientras que **standfordSTT** y **review_polarity** se concentran en el extremo inferior (mediana de 20 palabras). Esto resulta en un Dataset Unificado con alta dispersión, donde obtenemos una mediana de 133 palabras.

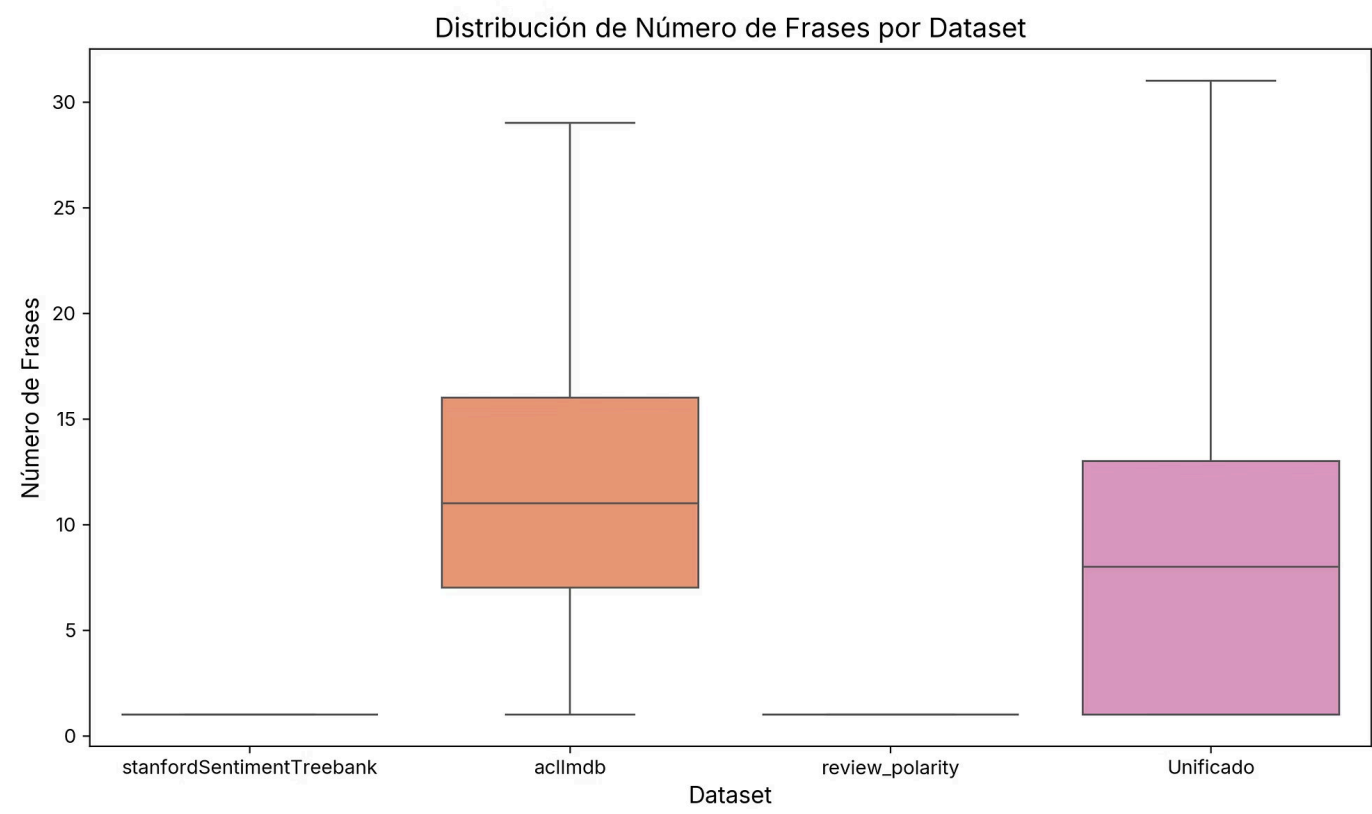
Distribución de caracteres



	aclImdb	review polarity	stanfordSTT	Unificado
Media	1310,6	114,3	103,8	963,5
Std	990,8	51,2	50,8	997,6
Mínimo	32,0	4,0	5,0	4,0
P25	699,0	76,0	64,0	168,0
Mediana	971,0	111,0	100,0	736,0
P75	1592,0	149,0	138,0	1260,0
P95	3394,0	205,0	194,0	2997,7
Máximo	13704,0	267,0	283,0	13704,0

En cuanto a los caracteres, hay una correlación directa con el número de palabras. **aclImdb** sigue como el único dataset con textos largos, mientras que los otros dos conjuntos son muy compactos y uniformes, sin superar nunca los 300 caracteres. Esto nos proporciona un dataset unificado más variado.

Distribución de frases



	aclImdb	review polarity	stanfordSTT	Unificado
Media	13,3	1,2	1,1	9,8
Std	9,3	0,5	0,3	9,6
Mínimo	1,0	1,0	1,0	1,0
P25	7,0	1,0	1,0	1,0
Mediana	11,0	1,0	1,0	8,0
P75	16,0	1,0	1,0	13,0
P95	32,0	2,0	2,0	28,0
Máximo	150,0	7,0	4,0	150,0

La diferencia estructural clave es la longitud: aclImdb consta de textos de párrafos (mediana de 11 frases), mientras que review_polarity y stanfordSentimentTreebank son colecciones de frases cortas (mediana de 1 frase). Esto hace que el *dataset* Unificado sea un híbrido con unidades simples y estructuras complejas de hasta 150 frases.

Conclusión de la longitud de reseñas

Concluimos que hay dos tipos de *datasets* diferentes en cuanto a longitud: *acllmbd* tiene textos largos y los otros dos son breves. Este desequilibrio en la longitud de las entradas (de 2 palabras a más de 2.000) puede ser un reto para el modelo de Machine Learning. Es posible que sea un punto crítico.

Análisis de Vocabulario y Ley de Zipf

Análisis de Vocabulario

Analizaremos el vocabulario para determinar su distribución y riqueza léxica, entendida como la variedad y complejidad del lenguaje que utiliza un autor o hablante. También identificaremos el "ruido" en cada *dataset* que no aporte valor a la predicción.

Estudiamos las métricas fundamentales de riqueza léxica y volumen de datos.

 **Tokens:** El número total de palabras en un texto.

Types: El número de palabras únicas.

TTR (Type-Token Ratio): Relación entre el número de palabras únicas y el total de palabras de un texto. Se utiliza para medir la riqueza léxica. El TTR puede ser alto o bajo: un valor alto indica una mayor variedad de vocabulario (con poca repetición de palabras), mientras que un valor bajo refleja un uso más limitado del léxico y una mayor repetición.

Hapax Legomena: Palabras que aparecen una sola vez en la totalidad del conjunto de datos.

	stanfordSST	review polarity	acllmbd	Unificado
Total Tokens	183.763	223.689	11.470.804	11.878.256
Type (Únicas)	17.412	21.381	390.931	394.370
TTR (Ratio Type/Token)	0,0948	0,0956	0,0341	0,0332
Hapax Legomena (Freq=1)	8.645 (49,6%)	11.013 (51,5%)	239.058 (61,2%)	239.624 (60,8%)

Concluimos que el conjunto de datos **acllmbd** es tan grande que aporta más del 96% de las palabras del texto unificado. Por lo tanto, cualquier decisión o tendencia del modelo estará fuertemente sesgada por este dataset.

Al analizar textos más amplios, el ingreso de palabras nuevas disminuye y se repiten las ya existentes (el ratio TTR cae de ~0,09 a 0,03). Además, el 61% de las palabras del vocabulario total aparecen una sola vez en todos los textos. Para un modelo predictivo, esto representa ruido e ineficiencia, no información útil.

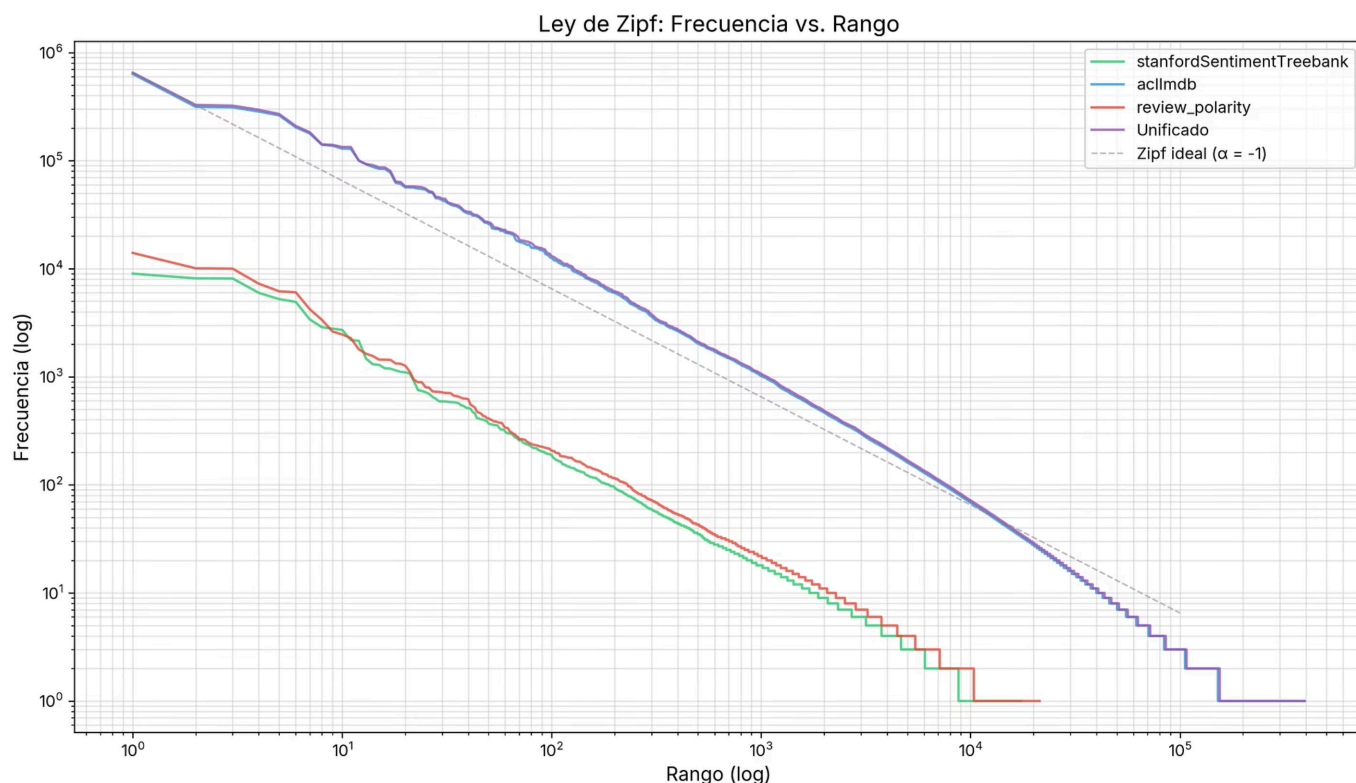
Las palabras más repetidas en el tope de la lista son exclusivamente artículos, preposiciones y signos de puntuación. Estas dominan el volumen del texto, pero no aportan ninguna pista sobre si la reseña es positiva o negativa.

Comprobación de la Ley de Zipf

Estudiaremos cuantas veces se repiten las palabras en los *datasets* y le asignaremos un valor en un rango, en función de las veces que ha aparecido. (lo visualizamos en un gráfico).

Calculamos la pendiente e intercepto para evaluar matemáticamente si las distribuciones de frecuencias de los *datasets* siguen la proporción inversa postulada por Zipf (cuya pendiente ideal en un gráfico log-log es de aproximadamente un: -1.0).

	review polarity	stanford Sentiment Treebank	acllmbd	Unificado
Pendiente de Zipf	-1,089	-1,094	-1,196	-1,204
Intercepto	4,585	4,520	6,480	6,529



Todos los conjuntos de datos presentan una estructura propia del lenguaje natural, pero revelan una distribución muy extrema: al tener pendientes ligeramente más pronunciadas que el ideal teórico, demuestran que un pequeño grupo de palabras vacías (artículos, preposiciones, conjunciones, signos de puntuación) acapara la inmensa mayoría del texto, mientras que se forma una gigantesca "larga cola" compuesta por miles de términos que aparecen una sola vez (*hapax legomena*). Debemos recortar los dos extremos del vocabulario para no saturar al modelo predictivo con ruido.

Decisiones a realizar

- **Eliminación de palabras vacías y puntuación:** Quitaremos las palabras que se repiten frecuentemente (artículos, preposiciones) y signos ortográficos que saturan el texto sin aportar información sobre el sentimiento de las opiniones.

Poda del vocabulario (*Trimming*): Eliminaremos la extensa larga cola de palabras raras para

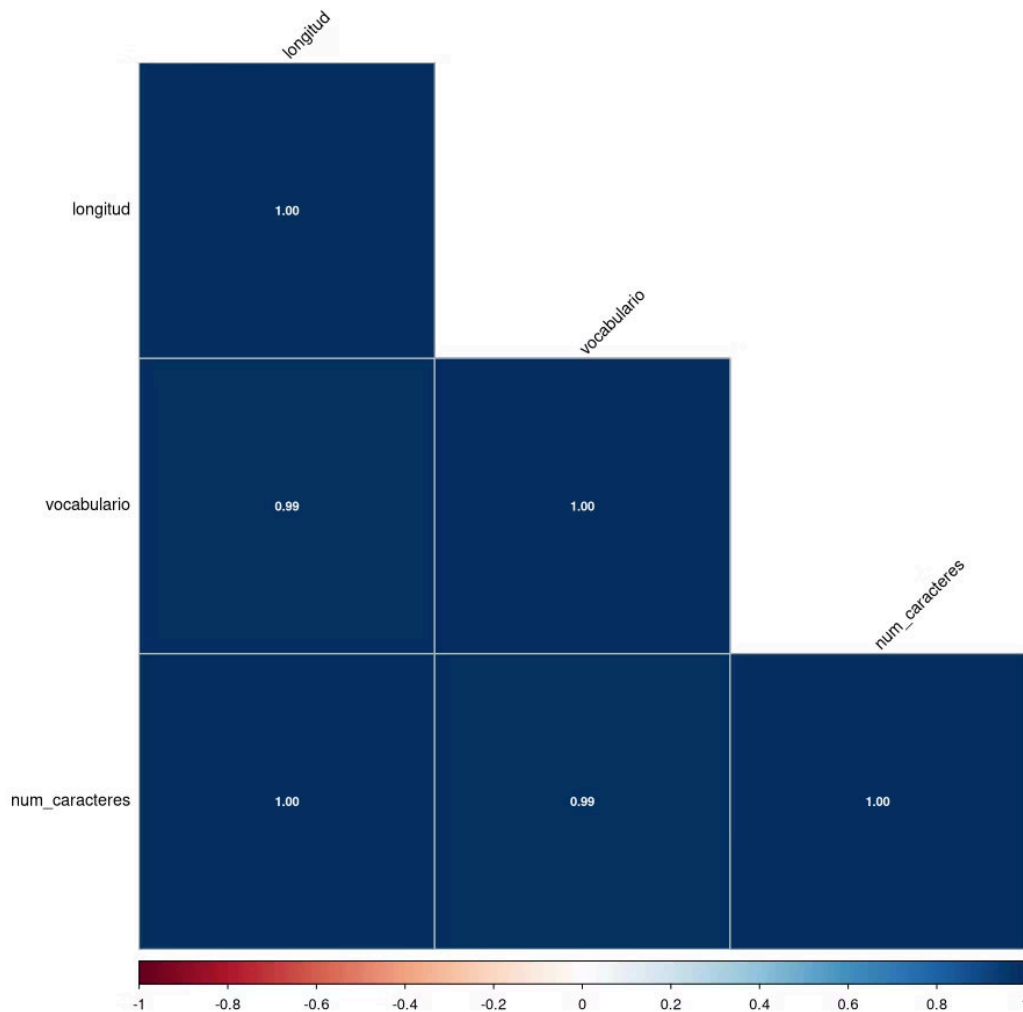
- descartar el ruido, ahorrar memoria y evitar que el modelo sufra de sobreajuste (*overfitting*).

Normalización del texto: Convertiremos todo el texto a minúsculas y transformaremos las palabras a su forma base para unificar términos gramaticalmente similares, reduciendo así el tamaño del diccionario y concentrando la señal predictiva.

3. Análisis Multivariante

Longitud vs Sentimiento

Correlación entre las variables

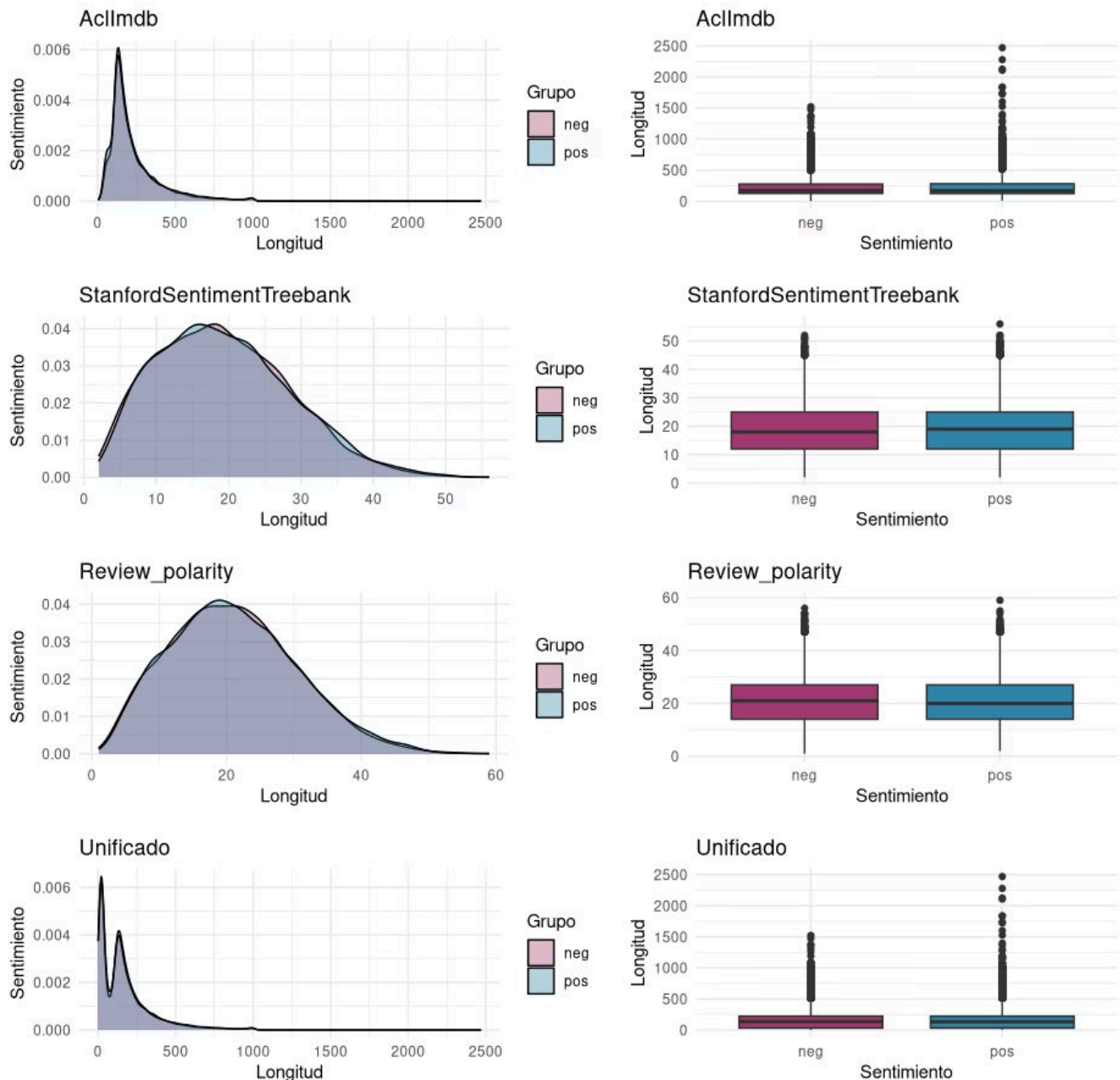


Los resultados indican que hay alta multicolinealidad entre las tres variables. Por tanto, realizaremos pruebas únicamente con la variable número de palabras que nos servirá de representación de la longitud.

Vamos a investigar si existe una relación entre la longitud de las reseñas y el sentimiento expresado.

Comparamos las distribuciones de longitud (número de palabras) entre reseñas positivas y negativas en cada *dataset*.

Comparación de distribuciones



Test de Mann-Whitney U

Como no hay normalidad, tendremos que realizar el test no paramétrico Mann-Whitney U para comprobar si hay diferencias significativas.

	acllmdb	review_polarity	stanfordSST	Unificado
Media Pos.	233,1	21,13	19,46	171
Media Neg.	229,6	20,96	19,14	169,6
Mediana Pos.	172,0	20	19	131
Mediana Neg.	174.0	21	18	135
U de Mann-Whitney	31.0652.184	14.010.333	11.140.190	613.882.594
P-valor	0,035	0,4906	0,2128	0,01993
¿Significativo?	No	No	No	No

Con un 99% de confianza el test nos asegura que no hay diferencias estadísticamente significativas en el sentimiento según la longitud de las reseñas. Pese a eso destacamos que en el dataset unificado es donde encontramos mayores diferencias (con un p-valor de 0,019).

Concluimos que la longitud por sí sola no es buen predictor del sentimiento.

Vocabulario Significativo

Vamos a estudiar qué palabras son características de cada clase.

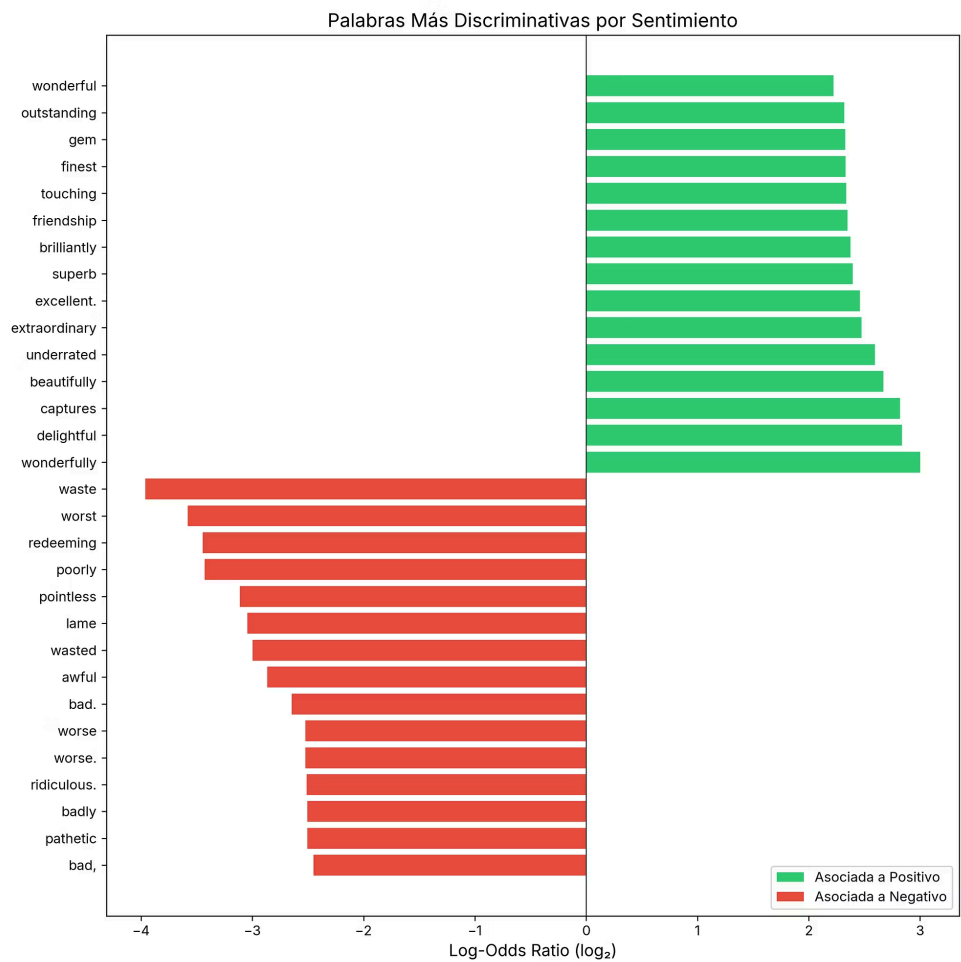
Frecuencias absolutas

Para ello, dividimos nuestros datos según el sentimiento y calculamos la frecuencia absoluta de cada término y su clase (identificando también si está en las dos clases).

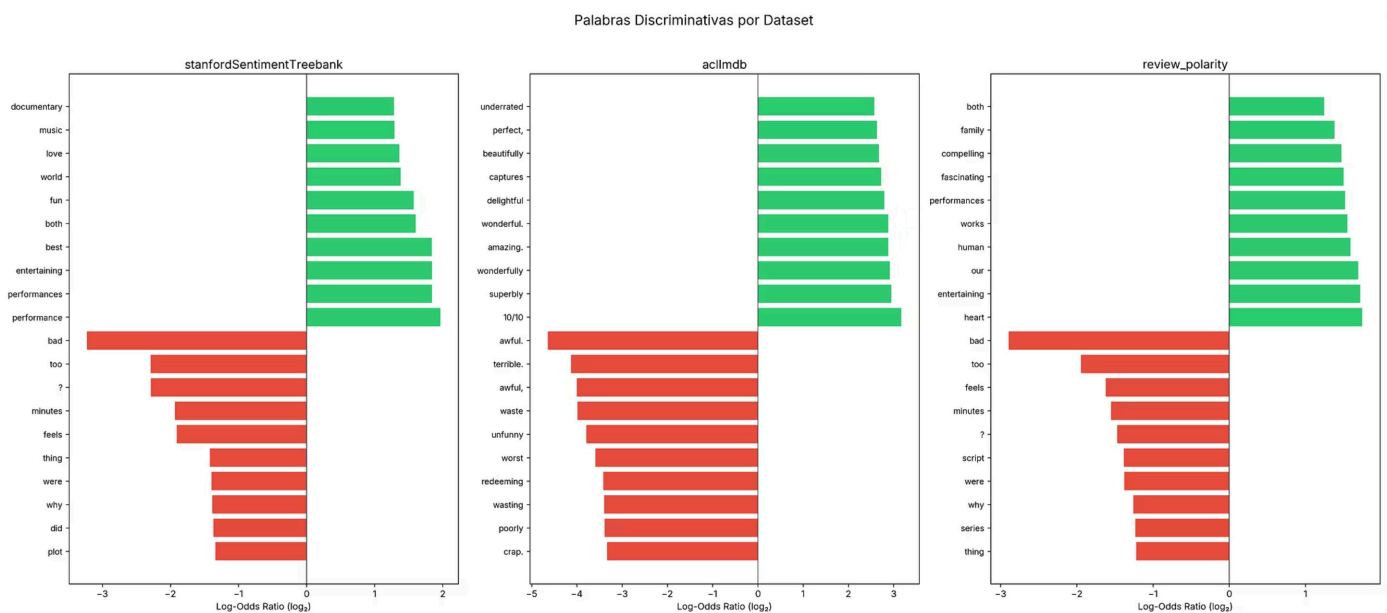
Total de palabras positivas	6.008.760
Total de palabras negativas	5.869.796
Vocabulario exclusivo positivo	147.111
Vocabulario exclusivo negativo	140.793
Vocabulario compartido	106.466

Log-Odds Ratio

Utilizamos los Log-Odds Ratio (LOR) para medir la fuerza de asociación de cada palabra con su sentimiento (calculamos la proporción de uso de la palabra en cada sentimiento). Además, aplicamos el suavizado de *Laplace* (sumar 1 a todas las frecuencias) para evitar divisiones entre cero. De tal manera que obtenemos en nuestro **Dataset Unificado**.



También comparamos cada *dataset* individual:



Prueba de relevancia

Para comprobar si cada palabra es significativamente importante en el sentimiento de la reseña aplicamos el Test *Chi-cuadrado*. Establecemos un nivel de confianza del 95%.

Para comprobar el tamaño del efecto de cada palabra sobre el sentimiento final utilizamos la V de Cramér.

 La **V de Cramér** es una medida del tamaño del efecto, nos dice qué tan fuerte es la relación entre una palabra y el sentimiento.

El Chi-cuadrado nos ayuda a detectar si existe asociación y la V de Cramér nos indica si esa asociación es débil, moderada o fuerte.

Establecemos los siguientes umbrales para interpretar la V de Cramér:

- $V < 0.01 \rightarrow$ Efecto nulo
- $V < 0.03 \rightarrow$ Efecto pequeño
- $V < 0.05 \rightarrow$ Efecto medio
- $V \geq 0.05 \rightarrow$ Efecto grande

Palabra	χ^2	p-valor	V de Cramér	Asociación
wonderfully	334.0	1.27e-74	0.0053	Nulo
delightful	242.1	1.40e-54	0.0045	Nulo
captures	251.5	1.25e-56	0.0046	Nulo
beautifully	436.5	6.40e-97	0.0061	Nulo
underrated	191.8	1.30e-43	0.0040	Nulo
extraordinary	168.5	1.56e-38	0.0038	Nulo
excellent.	171.9	2.88e-39	0.0038	Nulo
superb	405.5	3.55e-90	0.0058	Nulo
brilliantly	179.2	7.19e-41	0.0039	Nulo
friendship	185.1	3.68e-42	0.0039	Nulo

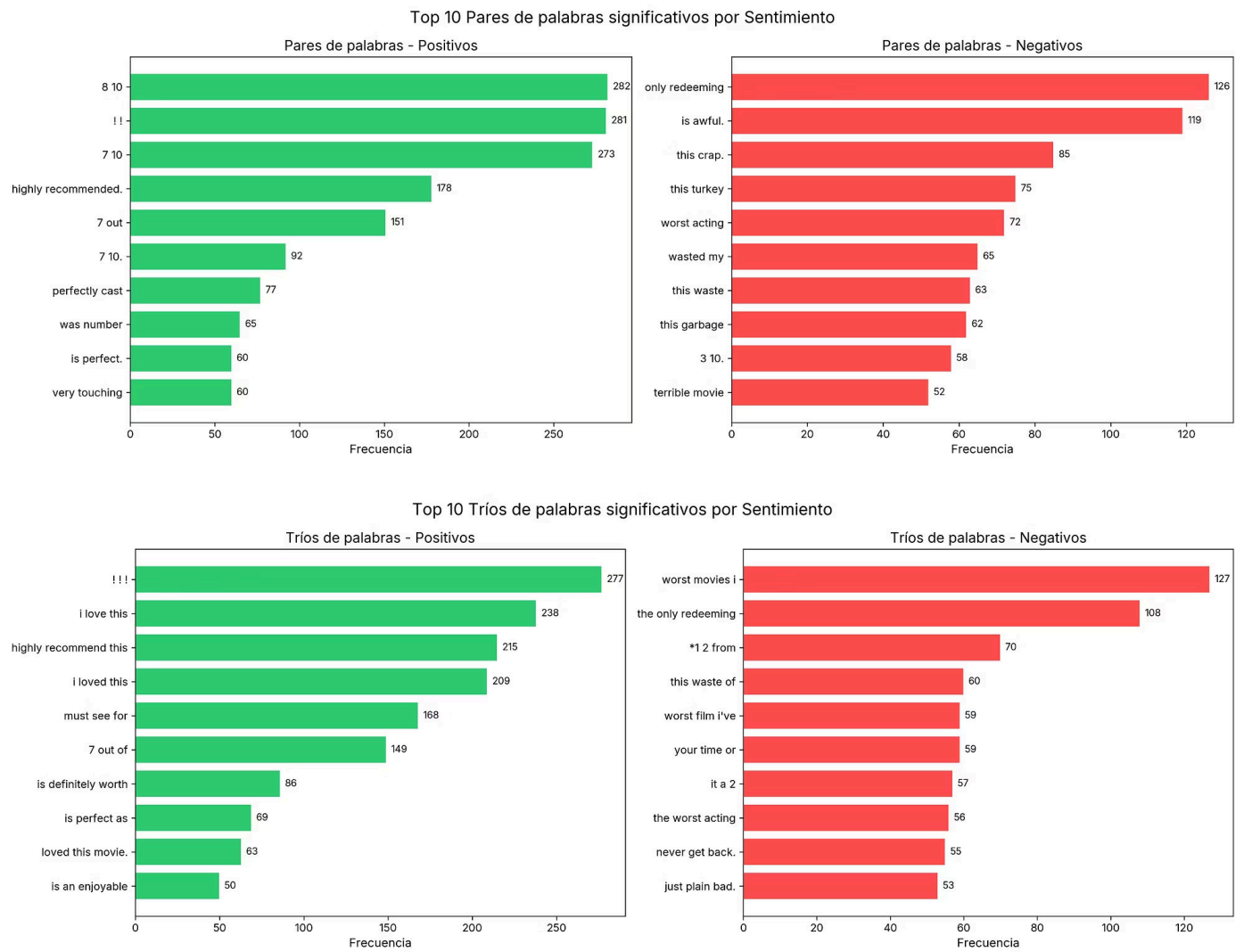
touching	307.9	6.28e-69	0.0051	Nulo
finest	210.8	9.09e-48	0.0042	Nulo
gem	197.1	9.06e-45	0.0041	Nulo
outstanding	253.7	4.02e-57	0.0046	Nulo
wonderful	1113.3	4.15e-244	0.0097	Nulo
portrait	138.6	5.50e-32	0.0034	Nulo
terrific	284.6	7.51e-64	0.0049	Nulo
magnificent	166.2	5.02e-38	0.0037	Nulo
excellent	1213.8	6.18e-266	0.0101	Pequeño
remarkable	202.5	5.95e-46	0.0041	Nulo
instead.	156.9	5.31e-36	0.0036	Nulo
terrible	1011.0	7.25e-222	0.0092	Nulo
stupid	1137.1	2.84e-249	0.0098	Nulo
crappy	232.4	1.83e-52	0.0044	Nulo
horrible	832.4	4.81e-183	0.0084	Nulo
bad,	769.9	1.91e-169	0.0081	Nulo
pathetic	320.9	9.30e-72	0.0052	Nulo
badly	531.8	1.17e-117	0.0067	Nulo
ridiculous.	168.1	1.97e-38	0.0038	Nulo
worse.	213.2	2.75e-48	0.0042	Nulo
worse	939.9	2.05e-206	0.0089	Nulo
bad.	903.4	1.80e-198	0.0087	Nulo
awful	1117.7	4.65e-245	0.0097	Nulo
wasted	592.7	6.62e-131	0.0071	Nulo

lame	603.1	3.54e-133	0.0071	Nulo
pointless	449.0	1.20e-99	0.0061	Nulo
poorly	885.1	1.68e-194	0.0086	Nulo
redeeming	428.4	3.59e-95	0.0060	Nulo
worst	3539.5	0.00e+00	0.0173	Pequeño
waste	2091.2	0.00e+00	0.0133	Pequeño

A pesar de tener muchas palabras que son significativas en el sentimiento de la reseña, destacamos que su impacto es nulo en la mayoría de casos, o pequeño (en *worst*, *waste* y *excellent*).

Análisis de pares y tríos de palabras

Estudiamos los pares de palabras y tríos de palabras significantes más frecuentes en cada clase para identificar patrones característicos del sentimiento (eliminando palabras vacías y signos de puntuación).



Cabe recalcar que a veces los usuarios dicen su puntuación sobre 10 y nos es útil para clasificar la reseña. Además de que también usan adjetivos superlativos para expresar su opinión.

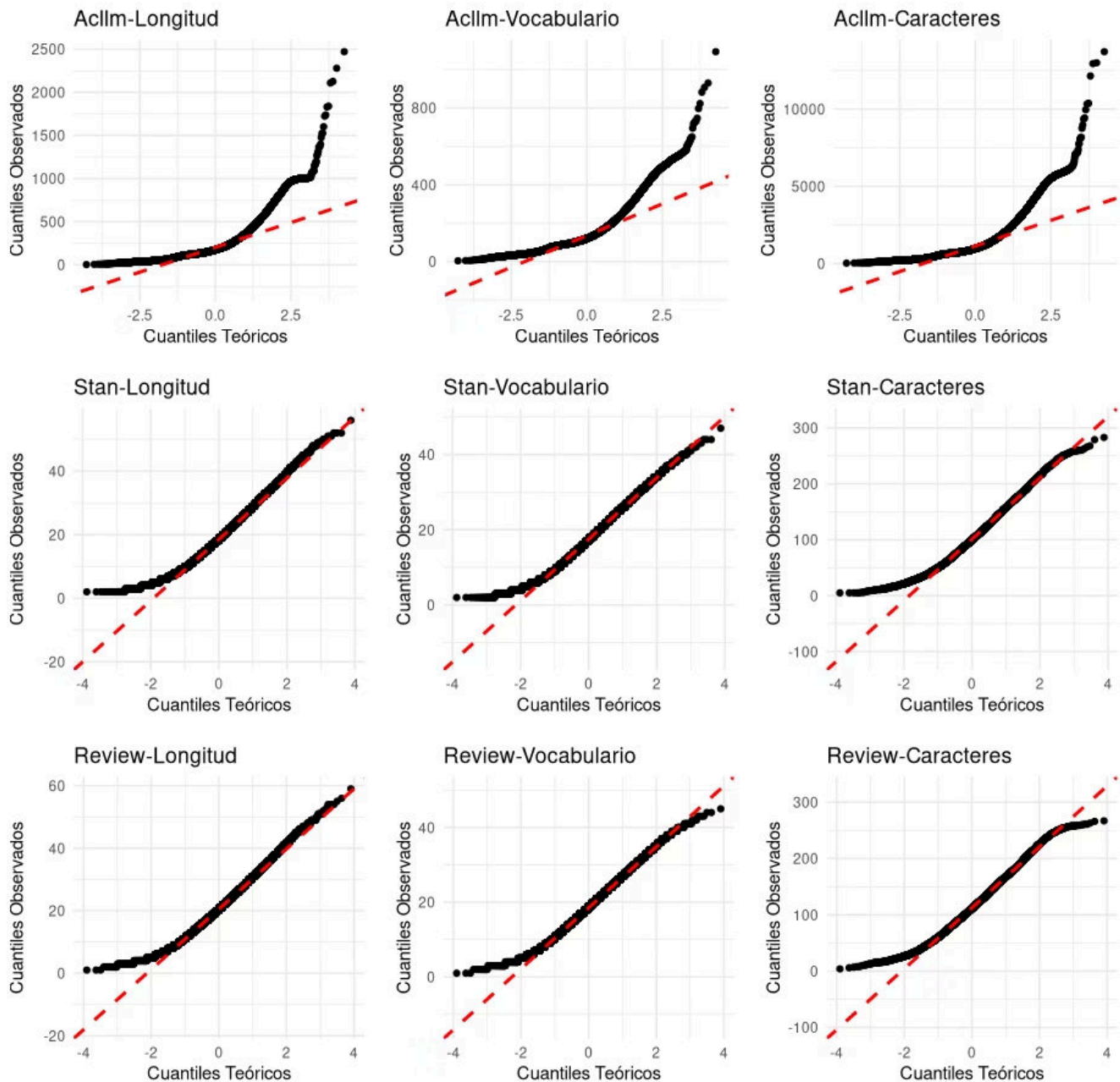
4. Evaluación de Supuestos

Normalidad de las Variables Numéricas

Comprobamos la normalidad en las variables numéricas mediante el test de Shapiro-Wilk. También visualizamos los gráficos Q-Q.

Dataset	Variable	W (Shapiro-Wilk)	p-valor	¿Normal?
acllmdb	N. Caracteres	0.77716	2.2e-16	No
acllmdb	Longitud Reseñas	0.78816	2.2e-16	No
acllmdb	Vocabulario	0.83576	2.2e-16	No
review_polarity	N. Caracteres	0.98577	2.2e-16	No
review_polarity	Longitud Reseñas	0.98532	2.2e-16	No
review_polarity	Vocabulario	0.99023	2.2e-16	No
stanfordSST	N. Caracteres	0.982	2.2e-16	No
stanfordSST	Longitud Reseñas	0.97847	2.2e-16	No
stanfordSST	Vocabulario	0.98474	2.2e-16	No

Q-Q plot Datasets

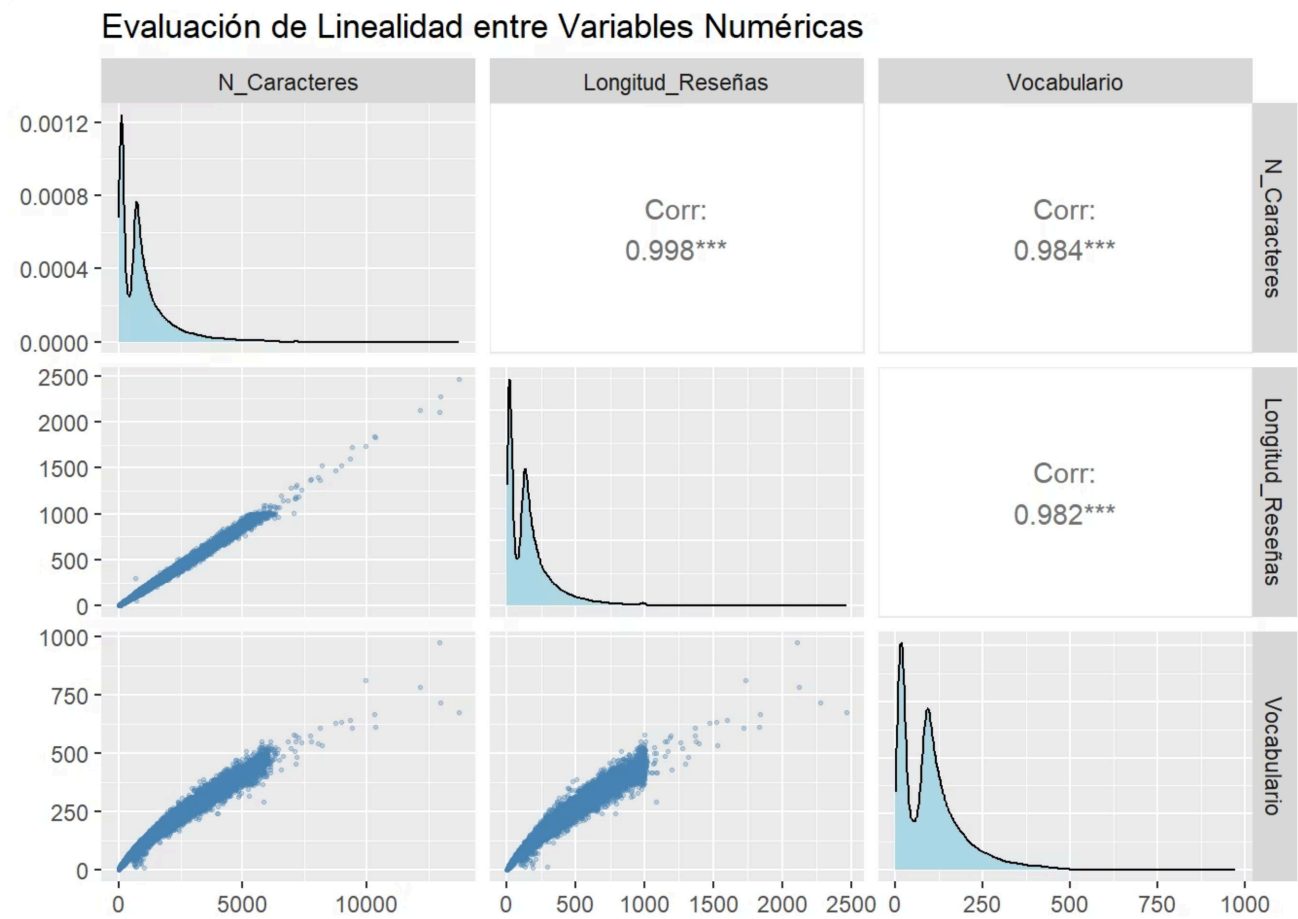


Ninguna de las variables numéricas sigue una distribución normal en ninguno de los tres datasets. Los p-valores son extremadamente pequeños (mucho menores que 0.01), lo que permite rechazar la hipótesis nula de normalidad. Los gráficos Q-Q confirman visualmente las desviaciones, especialmente en las colas de la distribución.

- El algoritmo k -NN no asume normalidad de los datos, por lo que este resultado no es problemático. Sin embargo, la ausencia de normalidad refuerza la necesidad de usar tests no paramétricos como *Mann-Whitney U* (ya usados). Además, la fuerte asimetría nos sugiere que debemos normalizar las variables antes de aplicar *Doc2Vec* y k -NN

Linealidad entre variables

Para evaluar la linealidad en nuestro *dataset*, hemos usado la gráfica de matriz de dispersión de `ggpairs`. Como nuestro problema tiene una variable categórica, el gráfico de residuos vs. ajustados no es de utilidad (evalúa la linealidad de un modelo de regresión lineal).



Al estudiar la gráfica, vemos que la diagonal principal nos muestra las gráficas de densidad, las cuales confirman la no normalidad. Los coeficientes de correlación de Pearson nos muestran que hay una relación lineal positiva extremadamente fuerte entre las variables, lo cual indica que tenemos una alta multicolinealidad.

Los diagramas de dispersión muestran una buena linealidad en general, ya que los puntos forman una línea. Aunque las interacciones con la variable Vocabulario muestran una ligera forma de embudo, lo que nos indica que hay una heterocedasticidad.

Homogeneidad de Varianzas

Varianzas de variables del *dataset*

Variable	F-value (Levene)	p-valor	¿Homogéneas?
N. Caracteres	31.177	2.365e-08	No
Longitud Reseñas	20.671	5.464e-16	No
Vocabulario	13.547	0.0002329	No

La varianza de las variables es heterogenea.

 Deberemos normalizar las características antes del entrenamiento del modelo. Usaremos *Doc2Vec* para proyectar los textos en un mismo espacio vectorial, independientemente de su longitud u origen.

5. Detección de Valores Atípicos (Outliers)

Detección mediante IQR y Z-Score

Utilizamos el rango intercuartílico y el Z-Score para detectar los outliers de nuestro *dataset* unificado.

Variables	Q1	Q3	IQR	Límite inferior	Límite superior	Outliers IQR	Outliers Z>3
Palabras	31	223	192	-257*	511	3.691 (5,29%)	1.546 (2,22%)
Palabras únicas	27	149	122	-156*	332	2.521 (3,62%)	1.280 (1,84%)
Caracteres	168	1.260	1.092	-1.470*	2.898	3.812 (5,47%)	1.559 (2,24%)

Alrededor del 5% (según IQR) o el 2% (según Z-Score) son textos atípicos. No es una cantidad masiva que invalide el dataset, pero lo ajustaremos.

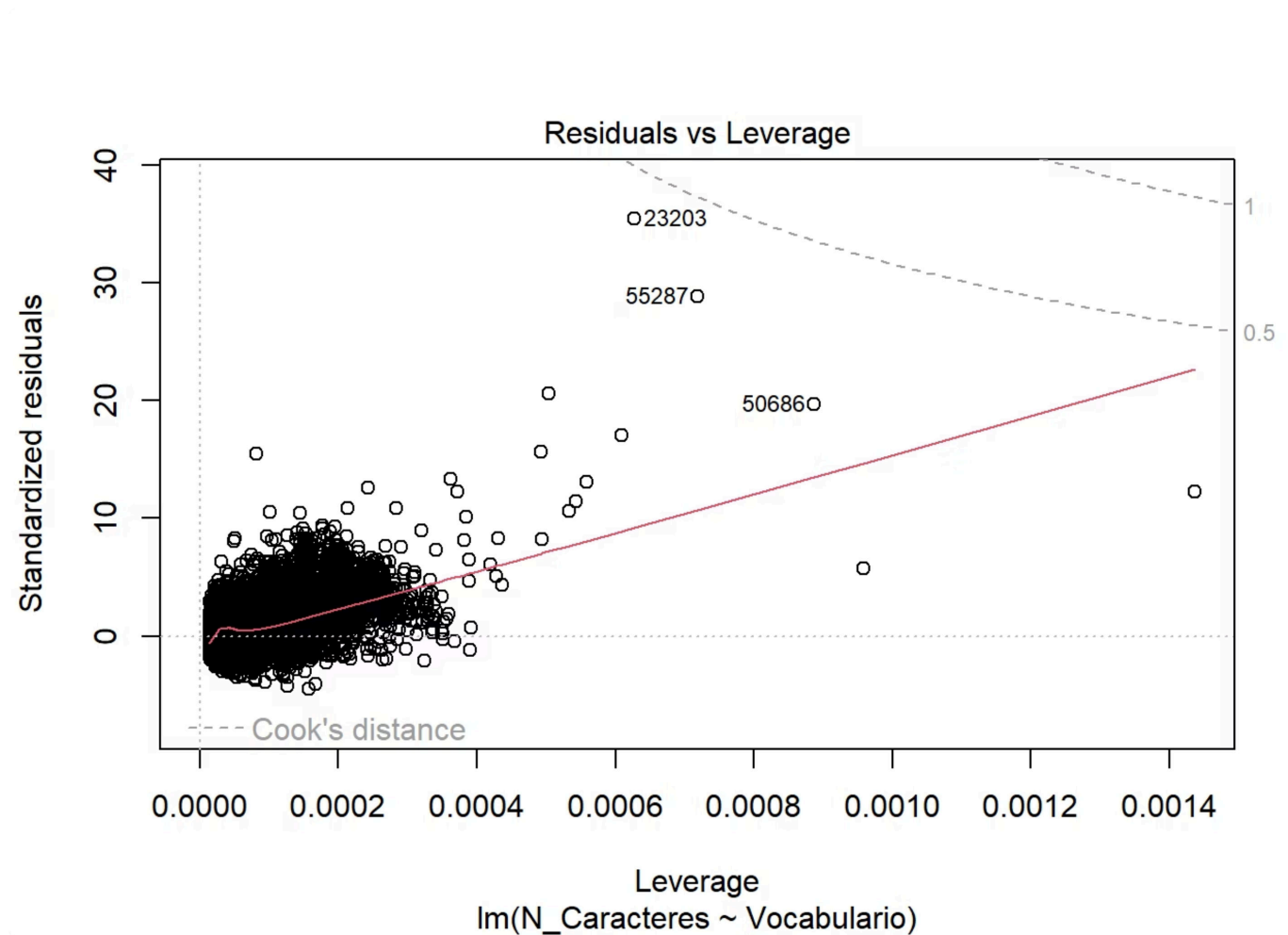
Límites negativos

Los límites inferiores negativos confirman que las anomalías son textos extremadamente largos o verbosos, ya que son los que desvirtúan el límite inferior y media.

Elegimos el método IQR en lugar del Z-score porque la distribución altamente asimétrica de los textos invalida a este último (como vemos en los valores negativos). Genera límites estadísticos irreales al verse sesgado por reseñas extremadamente largas que inflan su media.

Magnitud vs Apalancamiento

Para confirmar de manera visual los resultados obtenidos con el IQR. Usaremos la gráfica Magnitud vs Apalancamiento.



Observando la gráfica, vemos que nuestro modelo es estable debido a que no tiene valores atípicos influyentes. Esto se confirma ya que ningún punto supera las líneas que marcan la distancia de Cook.

Reseñas con Caracteres Anómalos

Analizamos el ratio de caracteres no textuales (espacios, puntuación, números, símbolos HTML) respecto al total de caracteres de cada reseña.

Métrica	Valor
Ratio medio	0,215 (21,5%)
Ratio mediano	0,214
Reseñas con ratio > 30%	506 (0,73%)
Reseñas con ratio > 50%	3 (0%)
Ratio máximo	0,873

El *dataset* tiene una excelente estructura, con un ratio medio normal del 21,5% de caracteres no textuales (normales de la redacción natural). La limpieza general es notable, apenas el 0,73% de las reseñas excede el umbral crítico del 30% de anomalías. Sin embargo, el valor máximo de 0.873 nos advierte que hay casos aislados con ruido extremo. Eliminaremos estos pocos registros para evitar que distorsionen las métricas geométricas del algoritmo K-NN.

Reseñas Extremas

Tipo	Cantidad	Porcentaje	Distribución sentimiento
Muy cortas (< 5 palabras)	417	0.6%	189 pos / 228 neg
Muy largas (> percentil 99)	695	1%	379 pos / 316 neg

Límites negativos

Los límites inferiores negativos confirman que las anomalías son textos extremadamente largos o verbosos, ya que son los que desvirtuan el límite inferior y media.

Elegimos el método IQR en lugar del Z-score porque la distribución altamente asimétrica de los textos invalida a este último (como vemos en los valores negativos). Genera límites estadísticos irreales al verse sesgado por reseñas extremadamente largas que inflan su media.

Reseñas con Caracteres Anómalos

Analizamos el ratio de caracteres no textuales (espacios, puntuación, números, símbolos HTML) respecto al total de caracteres de cada reseña.

Métrica	Valor
Ratio medio	0,215 (21,5%)
Ratio mediano	0,214
Reseñas con ratio > 30%	506 (0,73%)
Reseñas con ratio > 50%	3 (0%)
Ratio máximo	0,873

El *dataset* tiene una excelente estructura, con un ratio medio normal del 21,5% de caracteres no textuales (normales de la redacción natural). La limpieza general es notable, apenas el 0,73% de las reseñas excede el umbral crítico del 30% de anomalías. Sin embargo, el valor máximo de 0.873 nos advierte que hay casos aislados con ruido extremo. Eliminaremos estos pocos registros para evitar que distorsionen las métricas geométricas del algoritmo K-NN.

Reseñas Extremas

Tipo	Cantidad	Porcentaje	Distribución sentimiento
Muy cortas (< 5 palabras)	417	0.6%	189 pos / 228 neg
Muy largas (> percentil 99)	695	1%	379 pos / 316 neg

Las reseñas extremas constituyen una fracción marginal del dataset (apenas un 1,6% en conjunto), lo que muestra una gran normalidad en la longitud general de los textos. Las reseñas muy cortas (0,6%) muestran una leve tendencia hacia la negatividad. Por el contrario, el 1% de reseñas excepcionalmente largas se inclina hacia la positividad. De cara al modelo KNN, la estrategia óptima es eliminar los textos excesivamente cortos por su falta de contexto, y truncar (winsorizar) los muy largos para estabilizar el cálculo geométrico de distancias.

Decisión sobre el tratamiento de outliers

- **Eliminar registros sin contexto o con ruido:** Descartaremos las reseñas excesivamente cortas (menos de 5 palabras) por carecer de valor semántico suficiente para el análisis de sentimiento. Asimismo, eliminaremos los casos aislados con un ratio de caracteres anómalos superior al 30% para evitar distorsiones geométricas en el modelo.
- **Truncar reseñas extremadamente largas (Winsorización):** Para estabilizar el cálculo de distancias de K-NN sin perder información valiosa, pondremos un límite a los valores de los textos que superen el percentil 99.
- **Priorizar IQR sobre Z-Score:** Utilizaremos exclusivamente el Rango Intercuartílico (IQR) para definir los límites de truncamiento, ya que la distribución altamente asimétrica de los textos sesga la media y genera límites estadísticos irreales e inválidos si se emplea el Z-Score.

Valores ausentes (*missing*)

- ✓ Nuestro dataset, por su origen académico y didáctico no tiene valores ausentes. Será un problema que no tendremos que tratar.



6. Conclusiones del análisis exploratorio de datos

En nuestro análisis exploratorio de los tres *datasets* hemos concluido:

1. **Buen equilibrio de clases:** Las clases positiva y negativa están equilibradas, lo que elimina la necesidad de técnicas de sobremuestreo o submuestreo y facilita el entrenamiento del clasificador k-NN.
2. **Ausencia de normalidad, varianza heterogénea y multicolinealidad:** Los test estadísticos nos confirman que ninguna variable numérica sigue una distribución normal y que existe heterogeneidad en sus varianzas. Además, vemos una alta multicolinealidad entre las variables. Aunque K-NN no exige normalidad, esta fuerte asimetría y dispersión hacen obligatoria la normalización de las variables numéricas antes del entrenamiento del modelo.
3. **Vocabulario diferencial:** Existe una separación léxica entre clases pero cada una por si sola no es significativa. Necesitamos más información para determinar la clase.
4. **Tratamiento de Outliers:** Ejecutaremos una limpieza estructural eliminando registros sin contexto o con un ruido extremo. Para proteger el cálculo de distancias del algoritmo K-NN, truncaremos las reseñas excesivamente largas utilizando los límites del Rango Intercuartílico, ya que la asimetría de los datos invalida el uso del Z-Score.
5. **Necesidad de limpieza semántica:** Es necesario que apliquemos técnicas de procesamiento de lenguaje natural. Esto incluye la eliminación de palabras vacías y puntuación, y la normalización del texto (minúsculas) para unificar términos, reducir el diccionario y potenciar la señal predictiva.
6. **Uso de Doc2Vec:** Para reducir las diferencias superficiales de longitud y origen, y proyectar los textos en un espacio vectorial homogéneo, se utilizará *Doc2Vec* en conjunto con K-NN. Esto permitirá capturar el contenido semántico esencial para la clasificación.

Parte III. Desarrollo técnico y despliegue

Realizaremos la implementación práctica de nuestro sistema en forma de app. Entrenaremos y evaluaremos los modelos de clasificación predictivos, definiremos el flujo de los datos y diseñaremos una interfaz web interactiva para el usuario final. Todo esto quedará empaquetado de forma que sea fácil de usar.

1. Entrenamiento del modelo

Una vez unificados los *datasets* y realizado el análisis exploratorio de los datos, procedemos con el entrenamiento del modelo. Para esta tarea hemos decidido aplicar dos tipos de modelos:

- *K-Nearest Neighbors (k-NN)*: Como nos indica el libro de referencia.
- Regresión Logística: Probamos otro modelo con el objetivo de conseguir mejores métricas.

K-Nearest Neighbors (k-NN)

Se utiliza tanto para **clasificación** (predecir a qué grupo pertenece un dato) como para **regresión** (predecir un valor numérico).

A diferencia de otros algoritmos, *k-NN* no requiere de una fase de entrenamiento para construir un modelo. En su lugar, memoriza los datos y realiza los cálculos cuando se le presenta un nuevo dato.

Su funcionamiento se basa en calcular la distancia entre el nuevo dato y los datos existentes, seleccionando los *k* vecinos más cercanos (donde *k* es un valor que hemos elegido, en nuestro caso es 71). Para su clasificación, se aplica una votación entre estos vecinos: el nuevo dato se asigna a la clase mayoritaria (por ejemplo, si sus vecinos en su mayoría son reseñas positivas, se clasifica como positivo).

✔ Usaremos **K-Nearest Neighbors** en nuestro primer modelo

Diseñamos un modelo k-NN para la tarea de clasificación binaria de las reseñas (negativo vs. positivo).

Flujo de Preparación y Carga de Datos

El *pipeline* comienza con la carga de los conjuntos de datos previamente preprocesados utilizando la librería `pickle`.

Rutas de Origen:

- Características de entrenamiento: `data/processed/X_train.pkl`
- Etiquetas de entrenamiento: `data/processed/y_train.pkl`
- Características de prueba: `data/processed/X_test.pkl`
- Etiquetas de prueba: `data/processed/y_test.pkl`

Configuración del Modelo (Hiperparámetros)

Hemos instanciado un clasificador `KNeighborsClassifier` de la librería `scikit-learn`.

Hiperparámetro	Valor	Descripción Técnica
<code>n_neighbors</code>	71	Número de vecinos a considerar. Hemos elegido 71 tras muchas pruebas ya que reduce el riesgo de <i>overfitting</i> .
<code>weights</code>	"distance"	Los vecinos más cercanos tienen mayor influencia en la predicción.
<code>algorithm</code>	"brute"	Computación por fuerza bruta.
<code>metric</code>	"cosine"	Mide similitud angular entre vectores.
<code>p</code>	1	Parámetro de Minkowski
<code>leaf_size</code>	30	No afecta al rendimiento con <code>brute</code> .

Entrenamiento

Hemos entrenado el modelo con `.fit(X_train, y_train)` y exportado como archivo binario `.pkl` en `proyecto-package/dsr/knn_model.pkl`.

Evaluación

Para validar el modelo en `X_test`, hemos revisado tres aspectos clave:

1. Reporte de Clasificación:

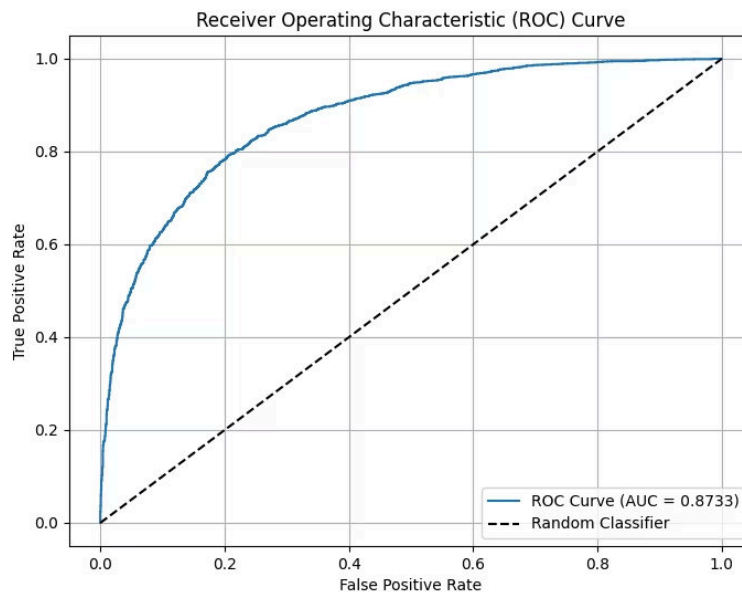
Evaluamos el rendimiento de las clases `negativo` y `positivo` de manera independiente mediante:

- **Precisión:** Capacidad del modelo para no etiquetar como positivo un caso que es negativo.
- **Exhaustividad (Recall):** Capacidad del modelo para encontrar todos los casos positivos.
- **F1-Score:** La media entre precisión y recall.
- **Accuracy General:** El porcentaje total de predicciones correctas sobre el test set.

2. Análisis de Capacidad Discriminatoria (AUC-ROC):

Implementamos:

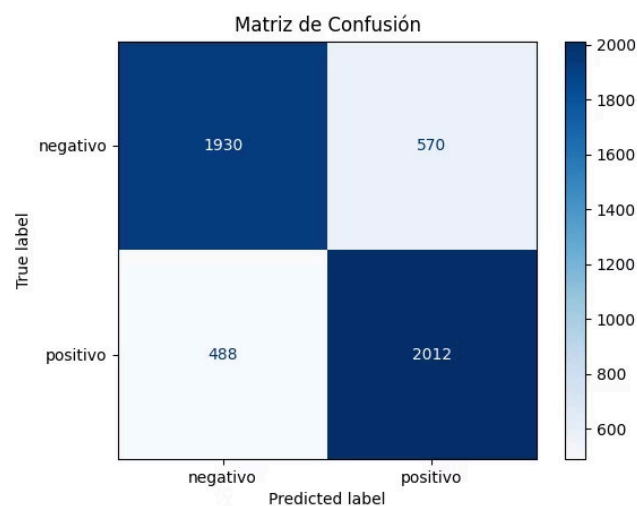
- **Métrica AUC-ROC:** Cuantifica la probabilidad de que el modelo ordene un caso positivo tomado al azar por encima de uno negativo.
- **Curva ROC:** Gráfica que contrasta la *Tasa de Verdaderos Positivos* frente a la *Tasa de Falsos Positivos* a lo largo de distintos umbrales de decisión.



3. Matriz de Confusión Visual:

Generamos un mapa de calor para inspeccionar visualmente la distribución de:

- **Verdaderos Negativos (TN)** y **Verdaderos Positivos (TP)** en la diagonal principal.
- **Falsos Positivos (FP)** y **Falsos Negativos (FN)** fuera de ella, permitiendo identificar sesgos o tendencias al error en alguna clase específica.



Regresión Logística

La regresión logística es un modelo de clasificación que transforma una combinación lineal mediante la función sigmoide para obtener probabilidades entre 0 y 1 para estimar la probabilidad de que una observación pertenezca a una clase.

 **Umbral de decisión:** Usamos un umbral de **0.5**: valores superiores se clasifican como **1** e inferiores como **0**.

A diferencia de la regresión lineal, no utiliza mínimos cuadrados, porque la forma sigmoide de su salida hace que ese enfoque no sea adecuado para la optimización. En su lugar, se entrena con **Entropía Cruzada Binaria** (*Log-Loss*), que mide el error entre las probabilidades predichas y las etiquetas reales. Los parámetros del modelo se ajustan mediante **Descenso de Gradiente**, buscando minimizar esa pérdida.

 Probamos con la **Regresión Logística** para tratar de lograr un mejor modelo.

Aplicando técnicas de regularización para asegurar una alta capacidad de generalización sobre datos no vistos.

Flujo de Carga y Consistencia de Datos

Al igual que en nuestro modelo anterior, utilizamos *pipeline* para mantener la consistencia de los datos mediante la carga de archivos estructurados con la librería `pickle`.

- Rutas de Origen:
 - Características de entrenamiento: `data/processed/X_train.pkl`
 - Etiquetas de entrenamiento: `data/processed/y_train.pkl`
 - Características de prueba: `data/processed/X_test.pkl`
 - Etiquetas de prueba: `data/processed/y_test.pkl`

Configuración del Modelo (Hiperparámetros)

Hemos instanciado el clasificador `LogisticRegression` de `scikit-learn`:

Hiperparámetro	Valor	Descripción Técnica
penalty	"l2"	Regularización Ridge. Añade una penalización cuadrática a la magnitud de los coeficientes para evitar que variables individuales dominen el modelo.
C	0.01	Inversa de la fuerza de regularización. Al ser un valor pequeño, está muy regularizado, forzando al modelo a mantener coeficientes pequeños y una estructura más simple.
solver	"saga"	<i>Stochastic Average Gradient Descent</i> . Es un optimizador avanzado y rápido para conjuntos de datos grandes, bueno para soportar penalizaciones L1 y L2 sin penalizar el rendimiento.
max_iter	1000	Límite máximo de iteraciones para que el optimizador <code>saga</code> converja hacia los coeficientes óptimos, garantizando estabilidad.

Entrenamiento

Entrenamos el modelo con `.fit(X_train, y_train)` y lo guardamos en `project-package/dsr/lr_model.pkl`.

Evaluación

La evaluación se organiza en tres partes:

1. Reporte de clasificación:

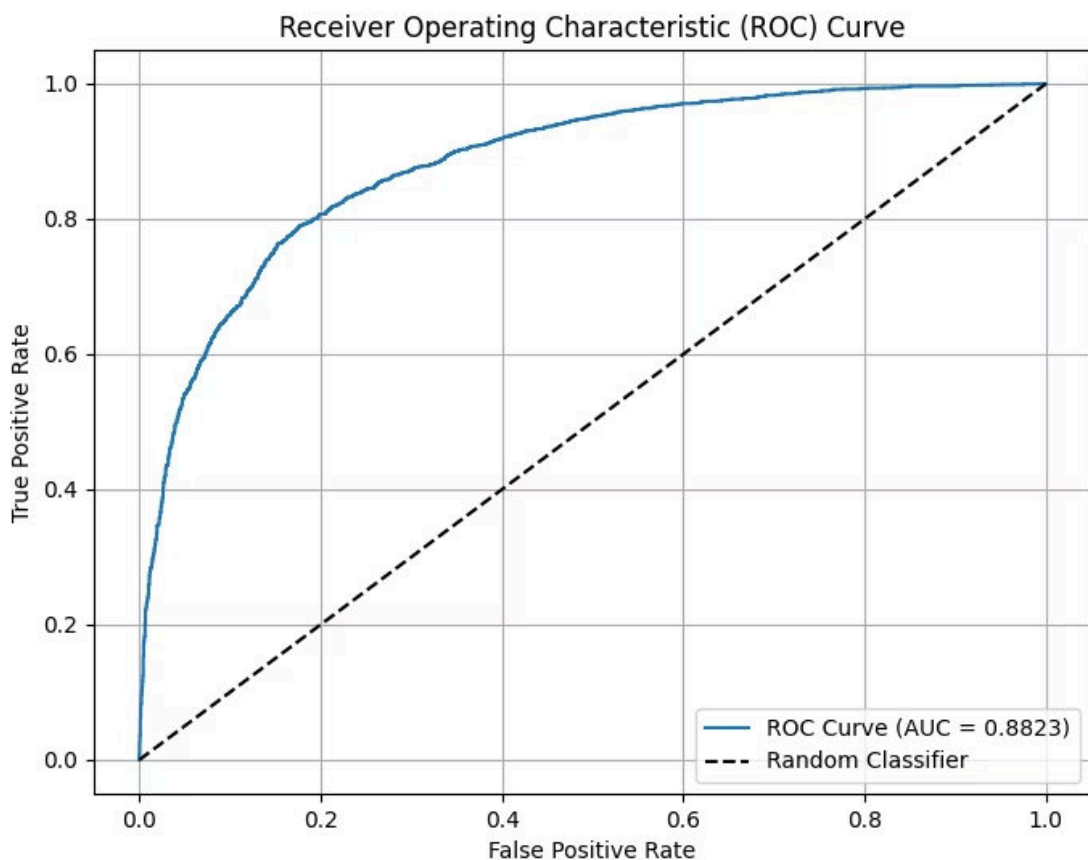
Hemos comprobado el rendimiento general e individual de las clases `negativo` y `positivo` a partir de:

- **Precision (Precisión):** El porcentaje de predicciones positivas correctas.
- **Recall (Exhaustividad):** La proporción de casos reales positivos que el modelo logró capturar con éxito.
- **F1-Score:** El balance métrico ponderado entre precisión y recall.
- **Accuracy:** Fracción global de aciertos del clasificador sobre el total de la muestra de prueba.

2. Análisis discriminatorio:

Utilizando las probabilidades calculadas mediante `.predict_proba()`, hemos extraído el vector de confianza de la clase positiva para crear:

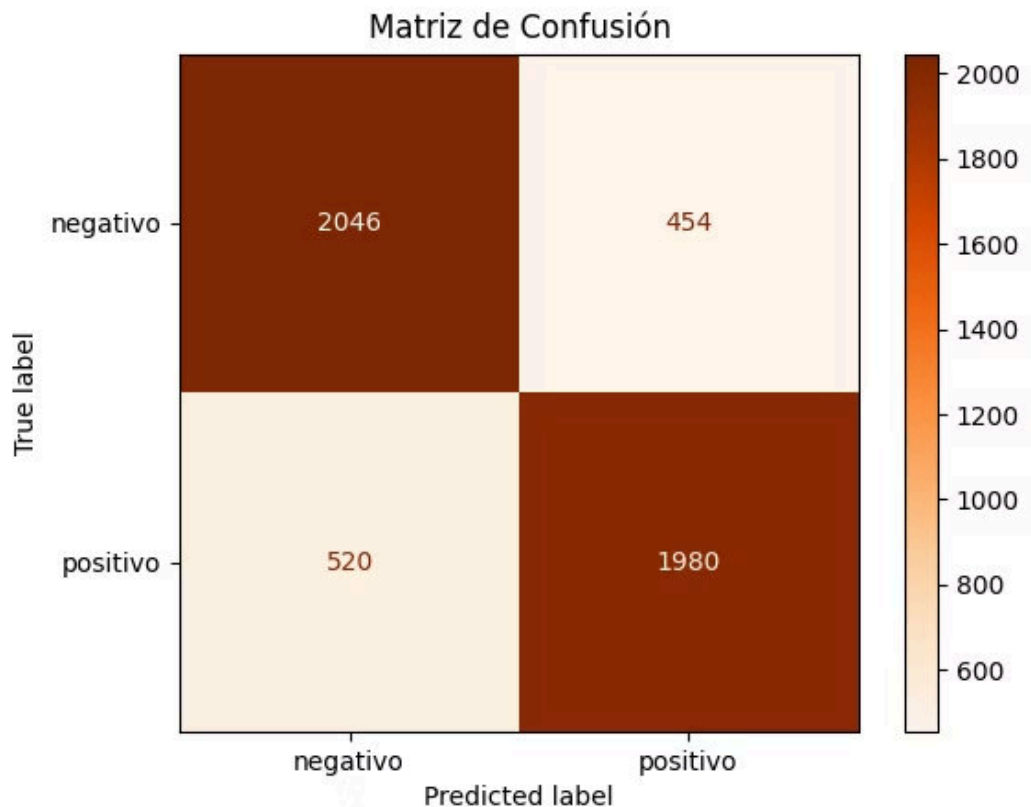
- **Métrica AUC-ROC:** Indica la robustez general de las probabilidades asignadas por el modelo, midiendo la probabilidad de que discrimine correctamente un caso positivo de uno negativo.
- **Curva ROC:** Visualización de la sensibilidad frente al ratio de falsos positivos en todos los umbrales de clasificación posibles, contrastada contra un clasificador aleatorio basal (línea discontinua).



3. Matriz confusión:

Hemos hecho un gráfico de un mapa de calor con las etiquetas `negativo` y `positivo`. Esta visualización nos permite analizar rápidamente :

- **Verdaderos Negativos (TN)** y **Verdaderos Positivos (TP)** en la diagonal principal.
- **Falsos Positivos (FP)** y **Falsos Negativos (FN)** fuera de ella, permitiendo identificar sesgos o tendencias al error en alguna clase específica.



Resultados de nuestros modelos

Al comparar el rendimiento de ambos algoritmos, observamos como el modelo de Regresión Logística supera al k -NN. Esto se refleja en el análisis de la curva ROC, donde el modelo logístico alcanza un AUC de **0.883** frente a un **0.873** obtenido por el k -NN.

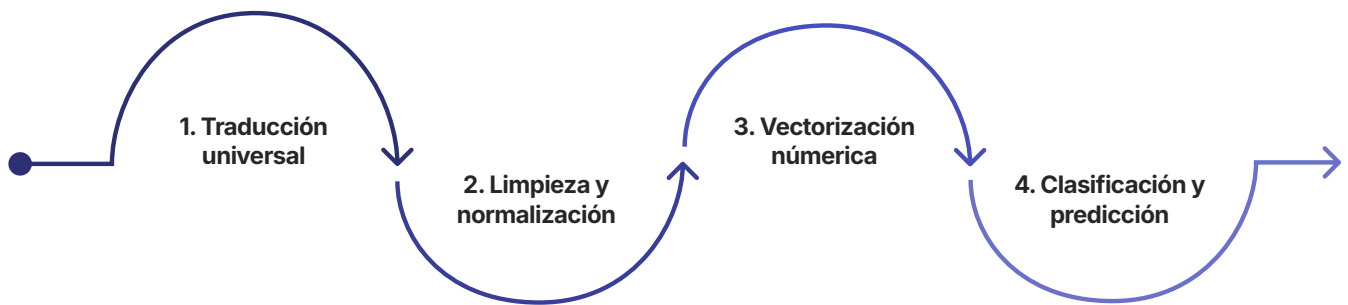
El AUC mide la capacidad del modelo para distinguir entre dos clases (si es negativo o positivo), por lo que nuestro modelo de Regresión Logística tiene mayor probabilidad de acierto en la clasificación. Esto quiere decir que hay una probabilidad del 88.3% de que clasifique correctamente el sentimiento. Mientras que con el modelo k -NN, esa probabilidad baja al 87.3%.

Por otro lado, la curva ROC evalúa el equilibrio entre la Sensibilidad (capacidad de detectar los casos positivos reales) y la Especificidad (capacidad de no dar falsos positivos). Que nuestro modelo logístico tenga mejor curva significa que, para cualquier nivel de dificultad el modelo seguirá encontrando más casos positivos reales que el k -NN. Por lo que es un modelo más robusto y estable.

2. Flujo de procesamiento de texto

Nosotros transformamos cada reseña o comentario en un texto normalizado y homogéneo para que nuestro sistema pueda analizarlo de forma consistente, sin depender del idioma original ni del formato de entrada.

Nuestro flujo de procesamiento de texto sigue los siguientes cuatro pasos:



1. Traducción Universal

Para que nuestra aplicación no dependa del idioma de entrada, el primer paso que realizamos es pasar la entrada del texto por un motor de traducción utilizando `GoogleTranslator` de la librería `deep-translator`. Detectamos automáticamente el idioma de origen mediante `source="auto"` y lo traducimos al inglés con `target="en"`.

Gracias a este paso, unificamos entradas de cualquier idioma y reducimos la variabilidad del texto antes del análisis. Así aseguramos que el resto de pasos trabajen sobre una representación común.

2. Limpieza y normalización

Una vez el texto está en inglés, nosotros lo pasamos por la clase `CleanText`, donde eliminamos el ruido que podría confundir a nuestro modelo.

Decodificación y filtrado web: Eliminamos etiquetas HTML y URLs. Estos elementos suelen aparecer en reseñas extraídas de páginas web y no forman parte del contenido real.

Depuración de caracteres: Suprimimos emojis, números y signos de puntuación. También reducimos letras repetidas de forma excesiva. Por ejemplo, `woooooow` se normaliza a `woow`. Así mejoramos la consistencia del vocabulario y evitamos que variaciones superficiales del texto confundan a nuestros modelos.

Tokenización y palabras vacías (*stopwords*): Convertimos el texto a minúsculas, lo dividimos en palabras individuales (*tokens*) y descartando palabras muy cortas y eliminamos las *stopwords*, palabras sin valor semántico como `the`, `and` o `is`. Eliminarlos mejora la claridad del análisis, porque nos quedamos con los términos que realmente aportan significado.

Etiquetado POS y lematización: Con `NLTK`, analizamos la función gramatical de cada palabra (sustantivo, verbo o adjetivo) y aplicamos `WordNetLemmatizer` para reducir cada token a su raíz léxica pura. Por ejemplo, `running` pasa a `run` y `better` se normaliza a `good`.

3. Vectorización numérica

Los modelos de *Machine Learning* no pueden leer palabras, sino números. Por eso, la lista de tokens limpios que obtenemos en la fase anterior la introducimos en nuestro modelo preentrenado `Doc2Vec`. Nosotros ejecutamos el método `infer_vector` para transformar el contexto y la semántica de la reseña en un único vector numérico que captura el significado global del texto de forma matemática.

Cada comentario se convierte en una representación numérica que contiene su contenido semántico. Codificamos palabras aisladas y la relación entre ellas dentro del texto, para que el modelo conserve el contexto relevante para la clasificación posterior.

4. Clasificación y predicción

Finalmente, el vector numérico lo transferimos al modelo predictivo seleccionado (`K-Nearest Neighbors` o `Regresión Logística`). El modelo elegido calcula dos valores: la clase binaria final (Positivo o Negativo) y las probabilidades matemáticas mediante `predict_proba`, que determinan el porcentaje de confianza del algoritmo en su predicción.

La clase final nos indica la orientación emocional de la reseña y la probabilidad nos permite expresar el grado de confianza del modelo.

3. Interfaz: Capturas y explicación del frontend desarrollado.

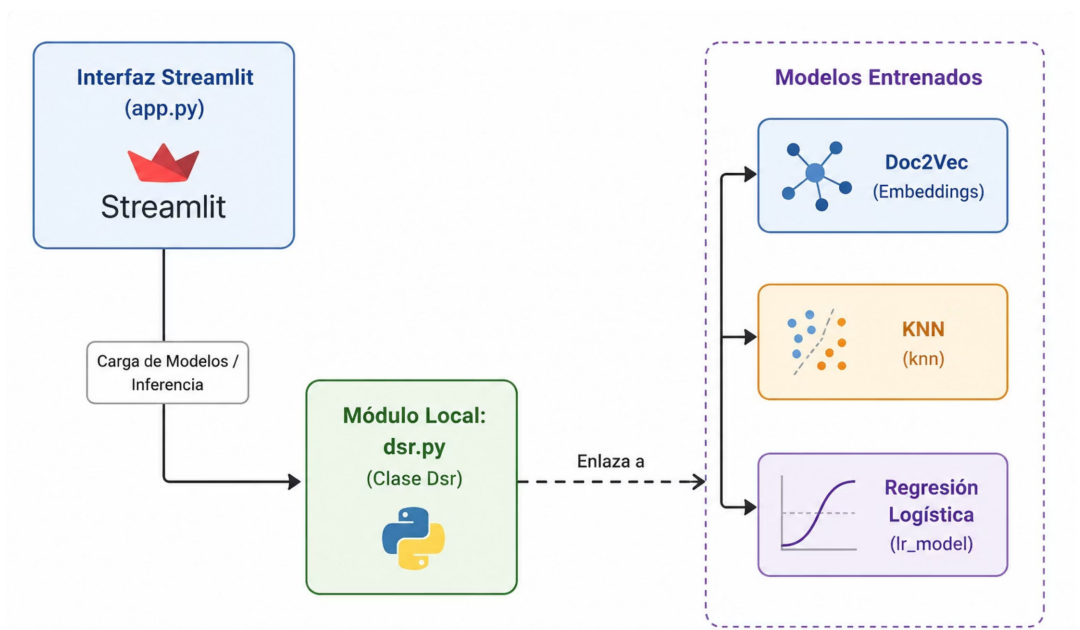
Para el diseño de la interfaz, realizamos varias lluvias de ideas para conseguir inspiración sobre como queríamos nuestra web interactiva. También recurrimos a herramientas de IA para que nos ayudase dándonos ideas, dando como resultado este diseño:



Creamos **DSR-AI**, una aplicación web interactiva diseñada para el análisis de sentimiento y la clasificación de reseñas de texto escritas por nuestros clientes. Desarrollamos la interfaz con el *framework* **Streamlit**, además, implementamos una capa de personalización estética con **CSS**. Nos inspiramos en un diseño minimalista al estilo de Google.

Arquitectura del Sistema

El software sigue una arquitectura desacoplada, en la que la interfaz de usuario (Frontend) se comunica directamente con el núcleo lógico predictivo (Backend) empaquetado localmente.



- `app.py`: Gestiona el estado de la aplicación, el renderizado de componentes web, la captura de inputs y la presentación de métricas.
- **Paquete `dsr`**: Es un módulo interno (`project-package`) que expone la clase `Dsr()`, encargada de vectorizar el texto de entrada y ejecutar los métodos `.predict()`.

Análisis de la Interfaz (Frontend)

1. Capa de Estilo y Personalización (CSS):

Para crear la capa de personalización, utilizamos un fondo oscuro y textos en gris claro. Además, añadimos un suavizado de bordes y aplicamos transparencias para un estilo moderno.

Incluimos un *Acerca de* que muestra el `README.md` del repositorio del proyecto.

2. Gestión del Flujo de Trabajo y Estado:

Para el correcto funcionamiento de la interfaz, hemos implementado un sistema de persistencia de datos en memoria utilizando la API de estado de Streamlit (`st.session_state`).

- ❗ La ejecución de Streamlit se basa en realizar *reruns* (re-ejecuciones) completos del script de Python cada vez que se detecta una interacción del usuario en la interfaz. Debido a esto, cualquier variable local del código se reinicia y pierde su valor original.

Para evitar la pérdida de información entre interacciones, hemos gestionado los siguientes estados críticos:

- `st.session_state.dsr`: Instancia y mantiene cargados en memoria los modelos predictivos desde el arranque de la sesión. Realizamos una única vez por sesión la costosa operación de cargar el modelo.
- `st.session_state.historial`: Almacenamos un diccionario para registrar el historial de cada sesión.

3. Modelos de Inteligencia Artificial

Nuestra plataforma permite la ejecución en tiempo real entre los dos modelos que hemos creado. Es decir, el usuario que utilice nuestra interfaz podrá seleccionar con cual modelo quiere trabajar.

- ✅ El texto se procesa en segundo plano y se transforma en vectores mediante *Doc2Vec* antes de ser utilizado por los modelos.

Pruebas de nuestra aplicación completa

Una vez competada la interfaz gráfica podemos probar al completo nuestra aplicación.

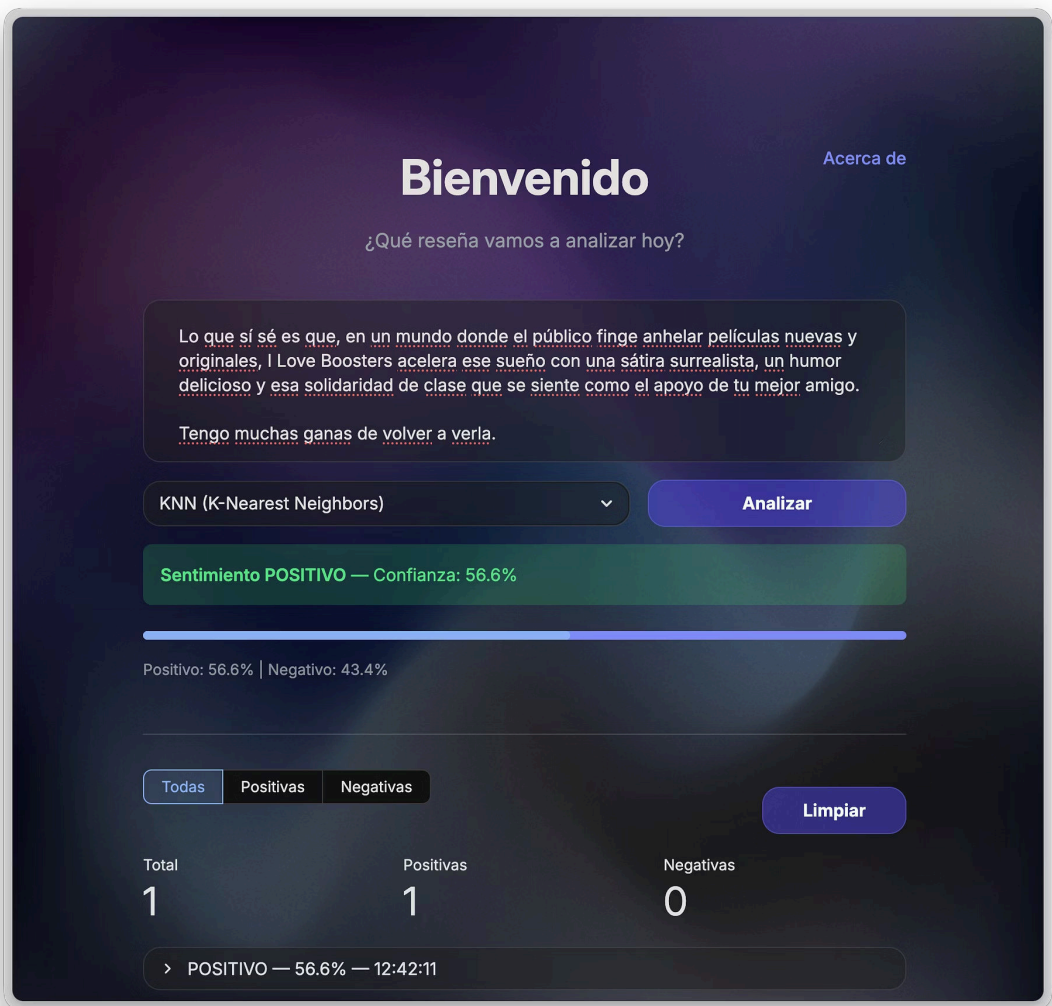
1. Reseña positiva

Boots Riley es un genio, pero no sé si la película a veces es demasiado inteligente para su propio bien o si mi propia estupidez y mi recién descubierto cinismo (y mis glúteos doloridos) empañaron una obra maestra.

Lo que sí sé es que, en un mundo donde el público finge anhelar películas nuevas y originales, I Love Boosters acelera ese sueño con una sátira surrealista, un humor delicioso y esa solidaridad de clase que se siente como el apoyo de tu mejor amigo.

Tengo muchas ganas de volver a verla.

Con modelo K-nn:



Con modelo de Regresión Logística:

Bienvenido

Acerca de

¿Qué reseña vamos a analizar hoy?

Lo que sí sé es que, en un mundo donde el público finge anhelar películas nuevas y originales, I Love Boosters acelera ese sueño con una sátira surrealista, un humor delicioso y esa solidaridad de clase que se siente como el apoyo de tu mejor amigo.

Tengo muchas ganas de volver a verla.

KNN (K-Nearest Neighbors)

Analizar

Sentimiento POSITIVO — Confianza: 56.6%

Positivo: 56.6% | Negativo: 43.4%

Todas

Positivas

Negativas

Limpiar

Total

Positivas

Negativas

1

1

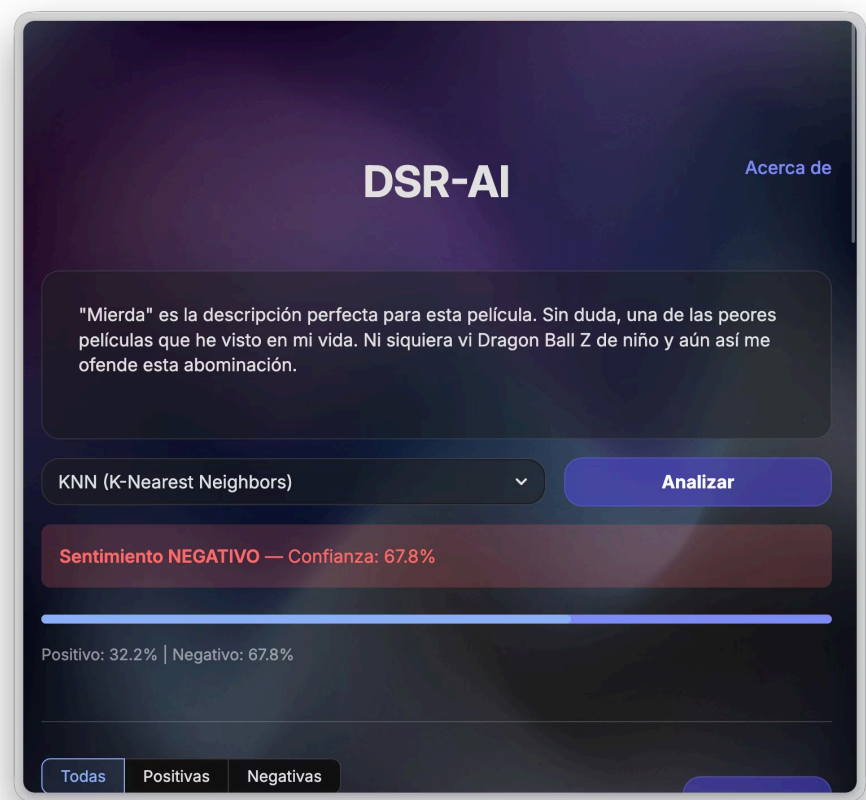
0

> POSITIVO — 56.6% — 12:42:11

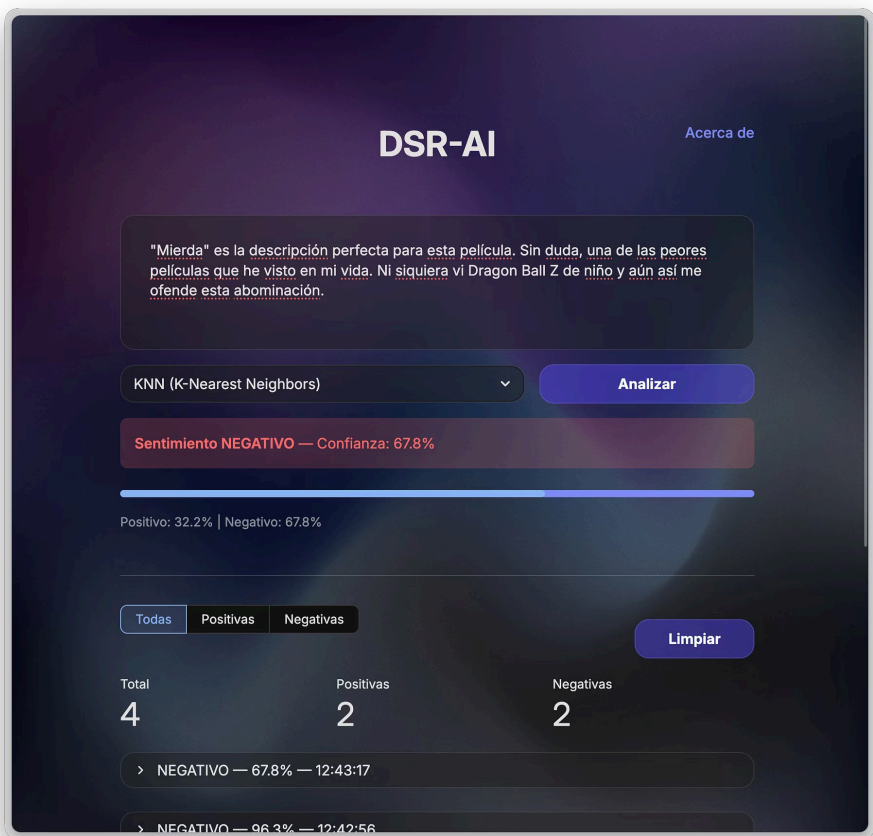
2. Reseña negativa

- "Mierda" es la descripción perfecta para esta película. Sin duda, una de las peores películas que he visto en mi vida. Ni siquiera vi Dragon Ball Z de niño y aún así me ofende esta abominación.

Con modelo K-nn:



Con modelo de Regresión Logística:



3. Historial de la sesión

TodasPositivasNegativas

Limpiar

Total	Positivas	Negativas
4	2	2

▼ NEGATIVO — 67.8% — 12:43:17

"Mierda" es la descripción perfecta para esta película. Sin duda, una de las peores películas que he visto en mi vida. Ni siquiera vi Dragon Ball Z de niño y aún así me ofende esta abominación.

Modelo: KNN | +32.2% / -67.8%

▼ NEGATIVO — 96.3% — 12:42:56

"Mierda" es la descripción perfecta para esta película. Sin duda, una de las peores películas que he visto en mi vida. Ni siquiera vi Dragon Ball Z de niño y aún así me ofende esta abominación.

Modelo: Regresión Logística | +3.7% / -96.3%

▼ POSITIVO — 93.2% — 12:42:34

Boots Riley es un genio, pero no sé si la película a veces es demasiado inteligente para su propio bien o si mi propia estupidez y mi recién descubierto cinismo (y mis glúteos doloridos) empañaron una obra maestra.

Lo que sí sé es que, en un mundo donde el público finge anhelar películas nuevas y originales, I Love Boosters acelera ese sueño con una sátira surrealista, un humor delicioso y esa solidaridad de clase que se siente como el apoyo de tu mejor amigo.

Tengo muchas ganas de volver a verla.

✔ Comprobamos que nuestra aplicación **funciona al completo** (*backend + frontend*).

4. Despliegue de la aplicación

Para el despliegue de la aplicación creamos un contenedor *Docker* (para permitir la reproducibilidad del proyecto). También lo subiremos a *DockerHub* para visualizarlo desde esta plataforma y lo empaquetaremos en un `zip` para el envío de la memoria.

Paquete de la app

Para el funcionamiento de la aplicación únicamente son necesarias las siguientes carpetas del repositorio:

- `data/`: Directorio con los modelos guardados y entrenados.
- `dsrc/` : Nucleo del código de la aplicación.
- `web-app/`: Implementación en streamlit (interfaz gráfica).
- `requirements.txt`: Librerías necesarias para el proyecto.

Docker

Definimos el siguiente *Dockerfile*.

```
FROM python:3.10-slim
WORKDIR /app
RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY data/ ./data/
COPY dsrc/ ./dsrc/
COPY web-app/ ./web-app/
COPY docs/screenshots/ ./docs/screenshots/
COPY README.md .
EXPOSE 8501
CMD ["streamlit", "run", "web-app/app.py", \
    "--server.port=8501", \
    "--server.address=0.0.0.0"]
```

De esta forma instalamos las dependencias y copiamos las carpetas necesarias.

❗ Utilizamos Python 3.10 debido a que la librería `gensim` (utilizada para vectorizar en Doc2Vec) no tiene soporte en versiones recientes.

Finalmente obtenemos un archivo `.tar` con la imagen del *Docker*.

La imagen del docker se exporto en un archivo `.tar` pero aparte se subio el contenedor a DockerHub. Como respaldo y para que el proyecto sea visible para todas las personas interesadas en probarlo.

DockerHub

Subimos la imagen del *Docker* obtenida a *DockerHub* para poder acceder desde esta plataforma.

✅ Es posible visualizar [aquí](#).

Si tenemos instalado *Docker Desktop* podemos correr la app con lo siguientes comandos:

```
docker pull adrac/dsr-ai:v1.0
docker run -p 8501:8501 adrac/dsr-ai:v1.0
```

Referencias

- Fundamentos de Doc2Vec: [Spot Intelligence - Doc2Vec Guide](#)
- Implementación práctica de Doc2Vec en NLP: [GeeksforGeeks - Doc2Vec in NLP](#)
- Documentación oficial de [Scikit-Learn - KNeighborsClassifier](#)
- Documentación oficial de [Scikit-Learn - LogisticRegression](#)
- Clasificación de texto mediante Regresión Logística: [GeeksforGeeks - Text Classification using Logistic Regression](#)
- Unfold Data Science: [Text Data Cleaning In Python](#)

Anexos

- [Repositorio del código fuente en GitHub](#)
- Entrenamiento de los modelos (archivo Jupyter Notebook .ipynb)
- Código fuente (archivo .zip)
- Imagen de *Docker* (archivo .tar)